

★ WEEK 8 · PART 1 of 3

State Space Models

A Hidden State, Glimpsed Through Noise

The two equations · the local level model · the framework behind everything

Part II opens with one unifying language. Behind the Kalman filter, structural models, and time-varying parameters lies a single idea: a hidden state you can only observe through noisy data.

1

PART 1 · STATE SPACE

The Unifying Framework

The hidden-state idea, the two equations that define every state space model, the local level model, and how familiar models like AR(1) are just special cases.

Shooting at What You Cannot See — Part 1

How a wartime gunnery problem birthed filtering

In World War II, gunners had to hit aircraft they could only track through noisy, jumpy radar. The true trajectory was hidden; the measurements were unreliable. How do you aim at what you cannot cleanly see?

The problem

- The target's true position and velocity were unobserved.
- Radar gave noisy, error-ridden snapshots.
- You needed the best estimate of the hidden truth, in real time.

The breakthrough

- Separate the SIGNAL (true trajectory) from the NOISE.
- Norbert Wiener's filtering theory tackled exactly this.
- It reframed tracking as estimating a hidden state.

Shooting at What You Cannot See — Part 2

The state space idea, and the lesson

Rudolf Kalman turned this into a recursive state-space solution in 1960 — the Kalman filter. It guided Apollo to the Moon, and it is the same idea we now apply to markets.

What it became

- A model of a hidden state evolving over time.
- Updated efficiently as each noisy observation arrives.
- The backbone of navigation, control, and finance.

The lesson for us

- Markets, too, hide their true state behind noisy prices.
- Trend, volatility, and hedge ratios are hidden states.
- State space gives us one language to estimate them all.

From a Toolbox to One Language

Where we are as Part II begins

1

Part I built a toolbox

ARIMA, GARCH, VAR, cointegration — a set of specialized models, each strong for its own kind of structure.

2

Part II offers a unifying framework

State space models express nearly all of them in one common form. The same two equations, with different matrices, recover each special case.

3

And it unlocks new power

Hidden states, time-varying parameters, missing data, and real-time updating — things a fixed model cannot do. This is the language of the Kalman filter (Week 9).

Part 1 in one line: one framework — a hidden state seen through noise — contains them all.

The Core Idea

A hidden state, observed only through noise

A state space model separates what you WANT to know (a hidden state) from what you can SEE (noisy observations). The state evolves over time; each observation is a noisy glimpse of it.

The hidden state

- The quantity you actually care about — a level, a trend, a ratio.
- It evolves through time by its own rule.
- You never observe it directly.

The noisy observation

- The data you actually record.
- It is the state plus measurement error.
- Your job: infer the state from these noisy glimpses.

The Two Equations

Every state space model is built from these

$$\begin{array}{ll} \text{state:} & x_t = T x_{t-1} + \eta_t \quad (\eta_t \sim \mathcal{N}(0, Q)) \\ \text{observation:} & y_t = Z x_t + \varepsilon_t \quad (\varepsilon_t \sim \mathcal{N}(0, H)) \end{array}$$

The state equation

- Describes how the hidden state evolves each period.
- The matrix T is the transition — the state's own dynamics.
- A random shock keeps the state from being deterministic.

The observation equation

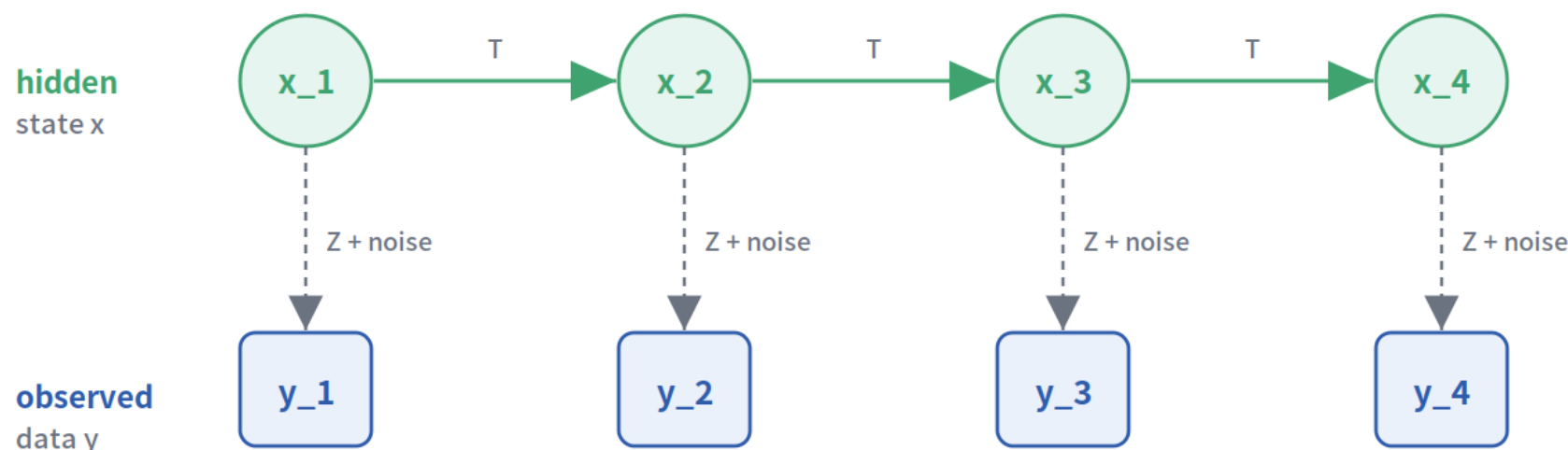
- Links the hidden state to the data you actually see.
- The matrix Z maps state to observation.
- Measurement noise separates the truth from the record.

The State Space Picture

A hidden chain of states evolves by the transition T ; each state emits a noisy observation through Z . We only ever see the bottom row — and infer the top from it.

The State Space Picture

A hidden chain of states evolves; we see only noisy snapshots of it.

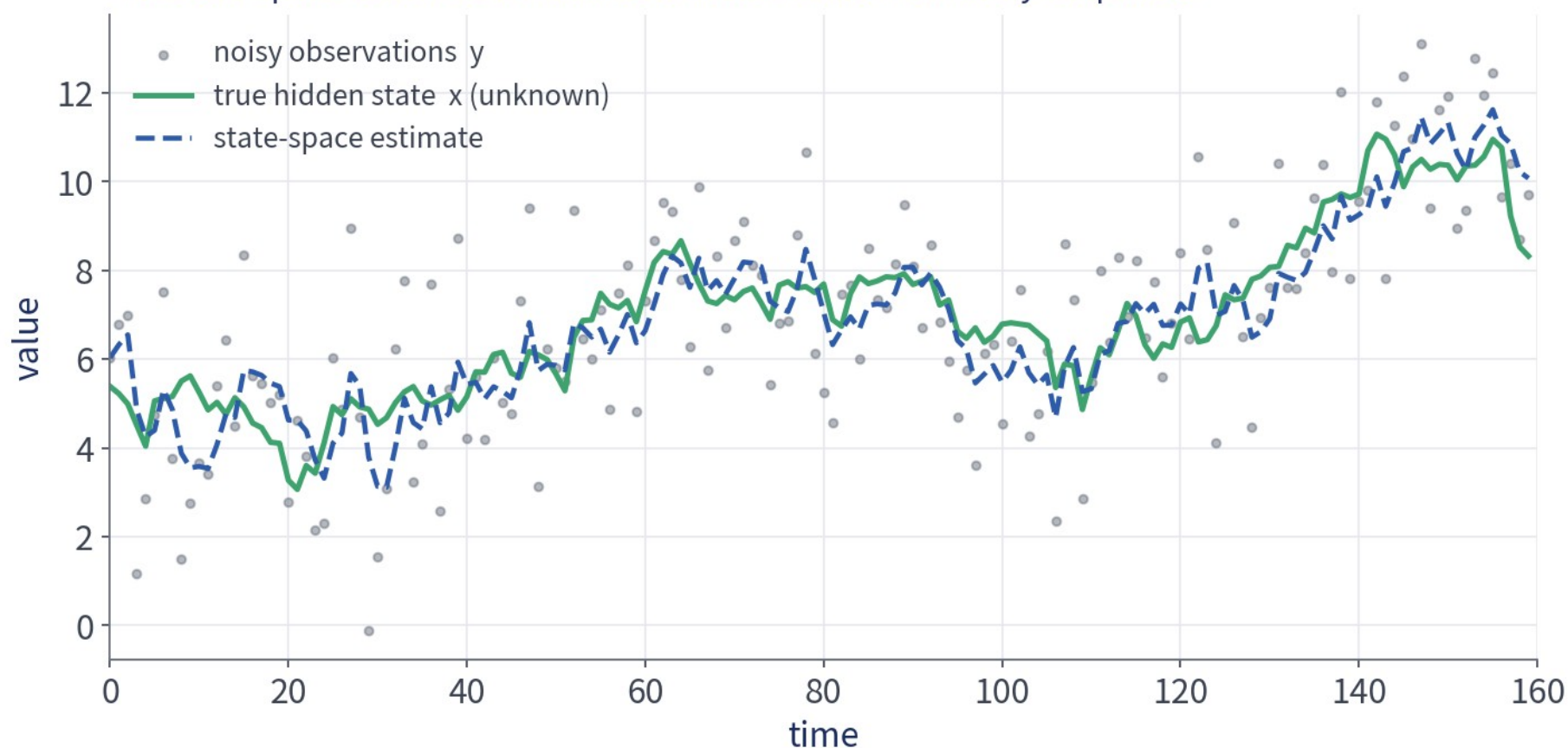


The state evolves by T (transition); each observation is Z times the state plus measurement noise. We infer x from the y 's.

Recovering the Hidden State

From a scatter of noisy observations, a state space model reconstructs a smooth estimate of the underlying hidden state — close to the true (unobserved) path.

The state-space idea: recover a smooth hidden state from noisy snapshots



The Local Level Model

The simplest state space model of all

$$\text{local level: } x_t = x_{t-1} + \eta_t, \quad y_t = x_t + \varepsilon_t$$

What it says

- The hidden level drifts as a random walk.
- Each observation is that level plus noise.
- It is 'a random walk seen through a fog'.

Why it matters

- It is the building block for everything richer.
- It already does something ARIMA cannot: extract a smooth level.
- Add a slope or a season and you get structural models.

Signal vs Noise

How much should you trust the latest data point?

signal-to-noise $q = \frac{\sigma_\eta^2}{\sigma_\varepsilon^2}$: large $q \Rightarrow$ trust data; small $q \Rightarrow$ smooth

The dial

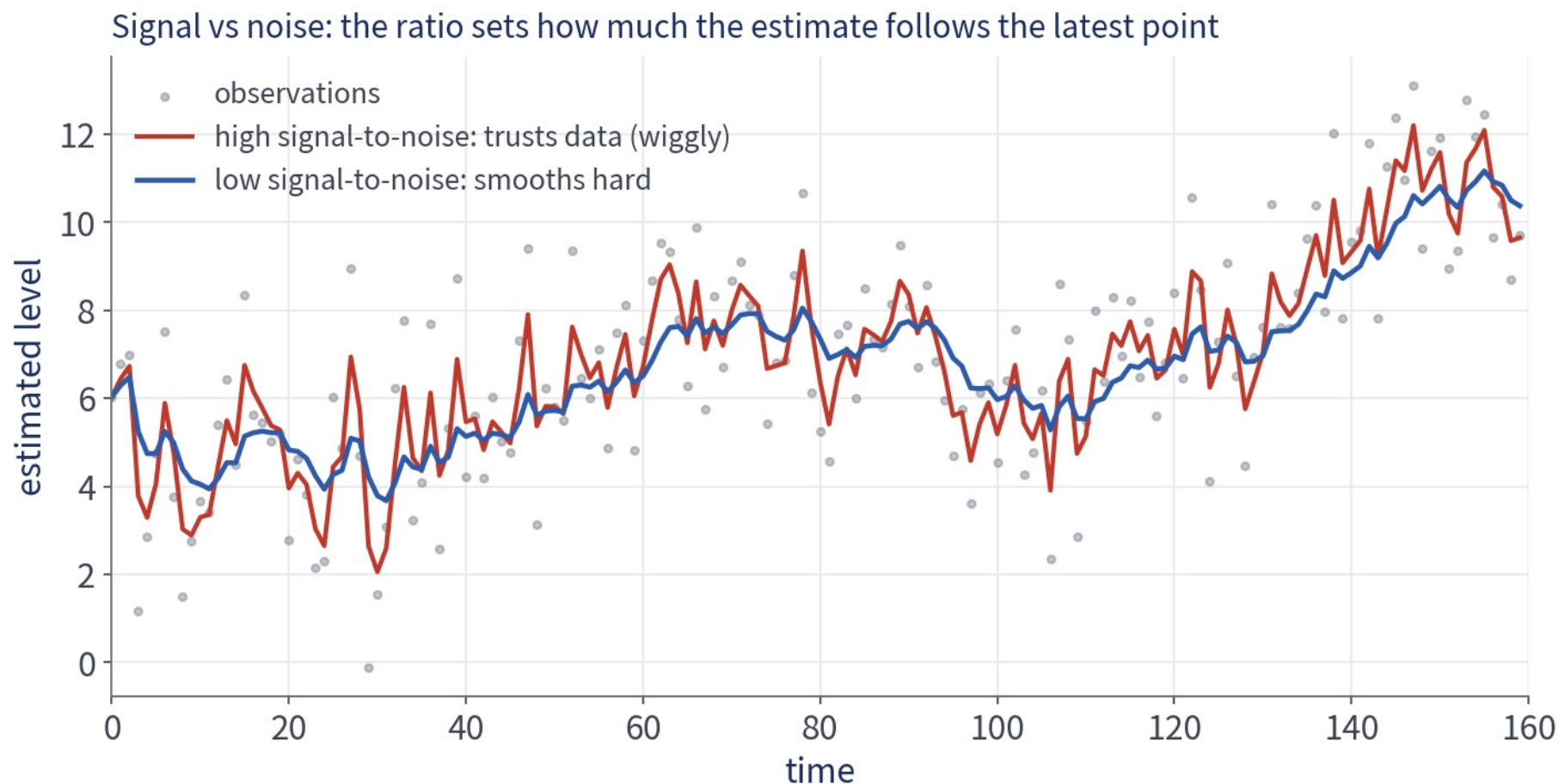
- The signal-to-noise ratio governs everything.
- High ratio: the state moves a lot — trust each new point.
- Low ratio: the state barely moves — smooth aggressively.

Why it is the key choice

- It decides how reactive vs stable your estimate is.
- It is estimated from the data, not set by hand.
- It is the same trade-off behind every filter (Week 9).

Trusting the Data, or Smoothing It

A high signal-to-noise ratio makes the estimate chase each observation (red); a low ratio smooths hard through the noise (blue). The ratio is the single most important choice.



ARIMA in Disguise: AR(1) as State Space

The familiar models are special cases

$$\text{AR}(1) \text{ as state space: } x_t = \phi x_{t-1} + \eta_t, \quad y_t = x_t$$

The mapping

- Let the transition be ϕ and the observation be exact.
- The state equation becomes an AR(1).
- No measurement noise — the state IS the series.

Why this is powerful

- ARIMA, ARMA, and more all fit this template.
- One estimation engine handles them all.
- You gain state space's extras for free — filtering, missing data.

One Framework Contains Them All

Why state space is the lingua franca of time series

Change the matrices T , Z , and the noise, and the same two equations become an AR, an ARIMA, a local trend, a structural model, or a time-varying regression. One form, many models.

What varies

- The transition matrix T sets the state's dynamics.
- The observation matrix Z sets how you see it.
- The noise variances set the signal-to-noise balance.

What stays the same

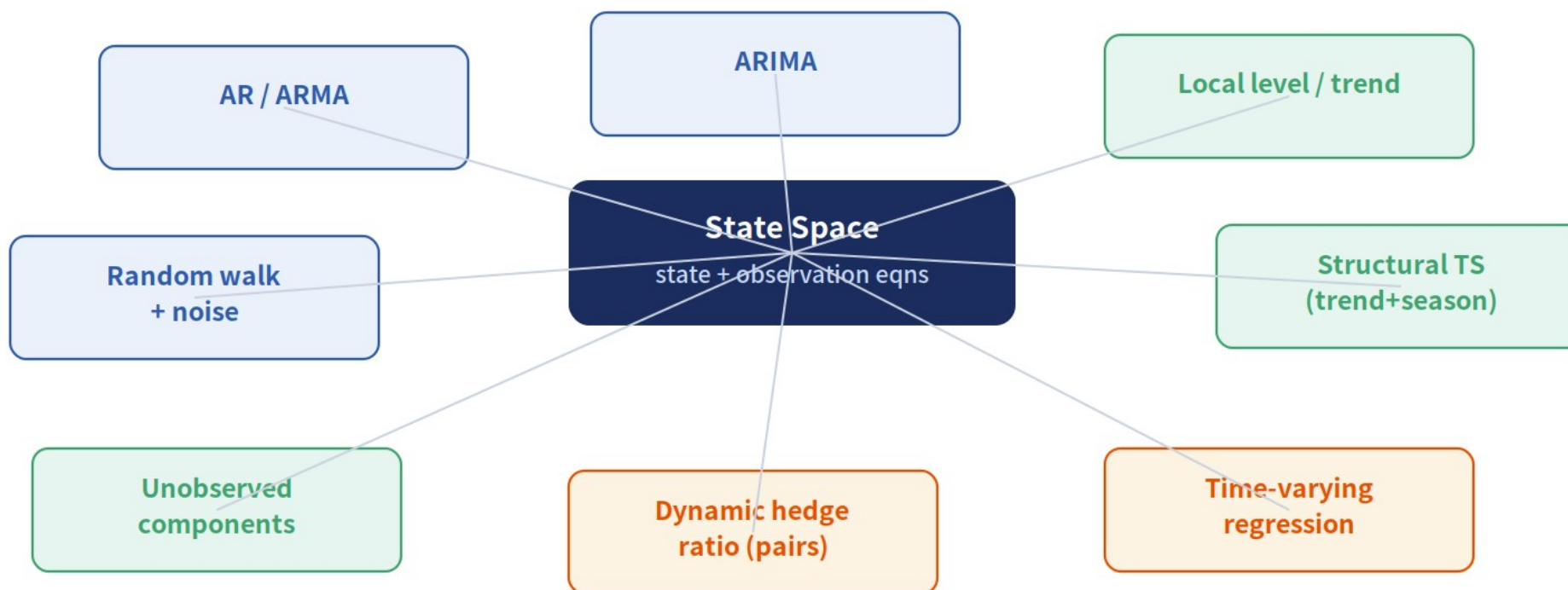
- The two-equation structure never changes.
- One estimation and inference engine covers all cases.
- Learn it once; apply it everywhere.

State Space as the Hub

AR/ARMA, ARIMA, local level and trend, structural models, time-varying regressions, dynamic hedge ratios — all are the same two equations with different matrices.

State Space as the Hub

Same two equations, different matrices - one framework contains them all.



Four Things State Space Buys You

Beyond what a fixed model can do

Capability	What it means
Hidden components	Extract an unobserved level, trend, or cycle directly.
Time-varying parameters	Let coefficients drift as the world changes.
Missing / irregular data	Handle gaps and uneven sampling naturally.
Real-time updating	Revise estimates the instant new data arrives.

Each of these is awkward or impossible in a fixed ARIMA — and routine in state space.

Time-Varying Parameters

Let the coefficients themselves drift

In a fixed model, a coefficient is one number for all time. In state space, a coefficient can BE a hidden state — free to evolve — capturing relationships that change with the regime.

The idea

- Treat a parameter as a state that follows its own random walk.
- The filter estimates its value at every point in time.
- The model adapts instead of assuming constancy.

Why it matters in finance

- Betas, hedge ratios, and risk exposures all drift.
- A fixed estimate is stale the moment the regime shifts.
- This sets up the dynamic hedge ratio (Part 2).

Missing and Irregular Data

Gaps are no obstacle

How it copes

- When an observation is missing, just skip the update step.
- The state still evolves; you simply don't correct it that period.
- Irregular timing is handled the same graceful way.

Why this is a big deal

- Real financial data has holidays, halts, and gaps.
- ARIMA needs a complete, evenly-spaced series.
- State space works around the holes automatically.

The Local Linear Trend Model

Add a slope state, and the model tracks trends

$$\text{local trend: } \mu_t = \mu_{t-1} + \beta_{t-1} + \eta_t, \quad \beta_t = \beta_{t-1} + \zeta_t, \quad y_t = \mu_t + \varepsilon_t$$

What's added

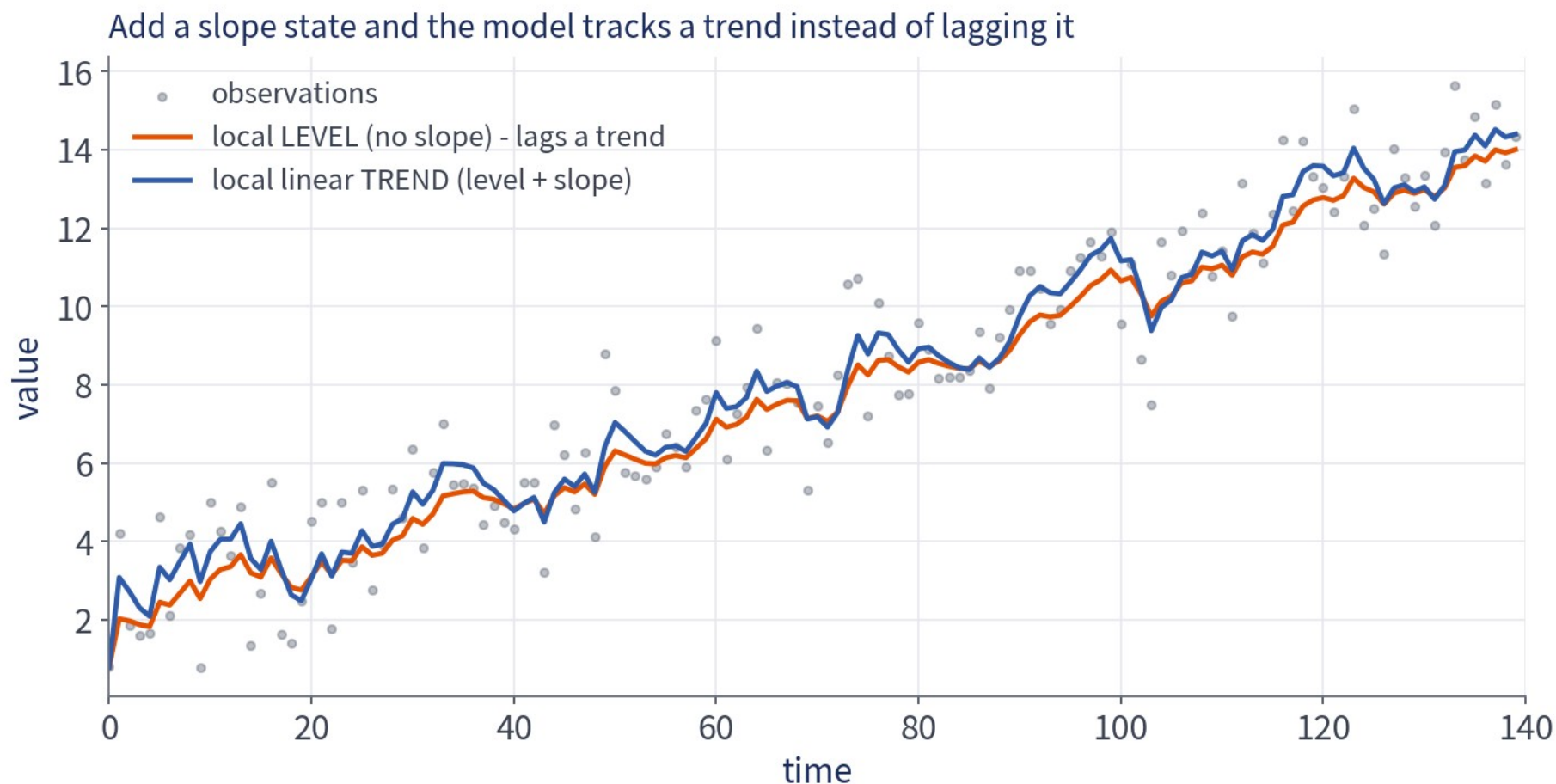
- A second state — the slope — evolves alongside the level.
- The level now moves by its own drift plus the current slope.
- Both are estimated in real time.

What it buys

- The model follows a trend instead of lagging behind it.
- It adapts as the slope itself changes.
- It is the workhorse of structural time-series decomposition.

Level vs Local Linear Trend

A local level alone lags a trending series (orange); adding a slope state lets the model track the trend as it moves (blue). Extra states buy extra structure.



Structural Time Series

Trend, season, and cycle — as states

The idea

- Give each feature its own state: level, slope, season, cycle.
- Stack them, and the model decomposes the series.
- Each component is estimated and can be read off directly.

Why it is appealing

- You SEE the trend and season, not just a forecast.
- Each part evolves and adapts over time.
- It is interpretable in a way ARIMA rarely is.

The State Space Workflow

How you actually use it in practice

1 · Specify

the states

Decide the hidden states: level, slope, season, or a parameter.

2 · Write eqns

T, Z, noise

Cast them in the two-equation form.

3 · Estimate

variances

Fit the noise variances by maximum likelihood.

4 · Filter

infer states

Run the filter to estimate the states over time.

5 · Use

read+forecast

Read components, forecast, and update live.

Steps 4–5 — the filtering that estimates the states — are the subject of Part 2 and Week 9.

Fit a Local Level Model

Direct Claude Code to extract a hidden level

GOAL Fit the simplest state space model to a noisy series, extract the hidden level, and read off the signal-to-noise balance.

[INPUT] Prompt · Claude Code

Write and run `local_level_lab.py` that:

1. loads a noisy series (e.g. a short window of daily prices, or a noisy macro series);
2. fits a local level model with `statsmodels.UnobservedComponents('local level');`
3. plots the data with the filtered and smoothed level estimates overlaid;
4. reports the estimated state and observation variances and the implied signal-to-noise.

Tell me whether the series is closer to a random walk or to pure noise around a level.

How to work with it

- Filtered uses data up to now; smoothed uses all data.
- A high signal-to-noise $\rightarrow$ the level tracks the data.
- A low ratio $\rightarrow$ a nearly flat, smooth level.
- Keep it; Part 2 explains filter vs smooth.

Deliverable: `local_level_lab.py` + a data-and-level plot with the variances.

Three Questions Before You Move On

If you can answer these, Part 1 has landed

Q1	What are the two equations that define every state space model?	State + observation
Q2	What does the signal-to-noise ratio control?	How much to trust new data
Q3	How can an AR(1) be written as a state space model?	Transition ϕ, exact obs

All three come from the two equations, the noise balance, and the unifying view.

Three Misconceptions to Avoid

Where state space intuition slips

Common myths

- "State space is a brand-new model to memorize."
- "The hidden state is just a smoothed version of the data."
- "You must set the signal-to-noise ratio by hand."

The reality

- It is a FRAMEWORK — your old models are special cases of it.
- The state has its own dynamics; smoothing is one output, not the idea.
- The noise variances are estimated from the data by likelihood.

Three Things to Carry Forward

Part 1 in one screen

1

A hidden state, seen through noise

Every state space model has two equations: how a hidden state evolves, and how noisy observations reveal it. The local level model is the simplest example.

2

One framework contains them all

Change the matrices and the same two equations become AR, ARIMA, local trend, or structural models — with hidden components and time-varying parameters as a bonus.

3

The signal-to-noise ratio is the key dial

It decides how much to trust each new observation versus smooth through it — the same trade-off that the Kalman filter will resolve optimally.

Next — Part 2: filtering, smoothing, and forecasting, and a drifting hedge ratio for pairs.

★ WEEK 8 · PART 1 · PRACTICE

State Space, Worked by Hand

Extract a Fair Value from Noise; Separate Level from Trend

The two equations, made concrete on real financial problems

Part 1's ideas, with the arithmetic shown. We write two financial problems in state-space form, then compute - a fair value pulled from a noisy print, and an index's level and momentum separated by hand.

1P

PART 1 · APPLICATIONS

Two Hidden States, Computed by Hand

Case A: the local level model pulls a fair value out of noisy prints in one filtering step. Case B: the local linear trend model splits an index into a level and a slope. Then a Claude Code lab.

What Part 1 Told Us

Now we make it concrete - with numbers

1

Two equations describe everything

An observation equation for what you see, and a state equation for the hidden thing that drives it.

2

The local level model

A hidden value drifts; you observe it through noise. The model separates the signal from the noise - we do one step by hand.

3

The local linear trend model

Add a slope, and the level can drift with momentum. We predict and update both, by hand, on a real index.

Everything here is a calculator exercise: two variances, one noisy print, one index surprise.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Write the problem in state-space form.
- Compute one step of the model by hand.
- Then read what the hidden state tells you.

What you should watch

- How much to trust a single noisy print.
- How a surprise updates a level and a slope.
- How one template fits many finance problems.

A Fair Value Hidden in Noisy Prints

The local level model, made concrete

A stock's true fair value drifts slowly, but every print you see is contaminated by noise - bid-ask bounce, thin liquidity, a stray trade. The local level model treats fair value as a hidden state and the print as a noisy observation. We compute the fair value from one print by hand

The two pieces

- Hidden state: the true fair value, drifting.
- Observation: the noisy printed price.
- Noise is larger than the drift here.

What we'll do by hand

- Write the observation and state equations.
- Compute the signal-to-noise ratio and gain.
- Turn a 100.60 print into a fair value.

Two Equations for Fair Value

What you see, and what is hidden

$$\begin{aligned} \text{observation: } P_t &= L_t + \varepsilon_t, & \varepsilon_t &\sim N(0, 0.25) \text{ (noisy print)} \\ \text{state: } L_t &= L_{t-1} + \eta_t, & \eta_t &\sim N(0, 0.04) \text{ (true value drifts)} \end{aligned}$$

The observation

- The print equals fair value plus noise.
- Noise variance is 0.25 - prices are noisy.
- You never see the fair value directly.

The state

- Fair value takes a small random step each day.
- Drift variance is only 0.04 - it moves slowly.
- This is the hidden thing we want.

Signal vs Noise

How much of a print is real?

Quantity	Value	Meaning
state (drift) variance	0.04	how fast fair value moves
obs (noise) variance	0.25	how noisy each print is
signal-to-noise q	0.16	drift / noise - noise dominates

A low q means most of a print's move is noise, not real - so a new print should move our estimate only a little.

One Filtering Step

From a 100.60 print to a fair value

$$\text{predict: } R = C + \sigma_{\eta}^2 = 0.10 + 0.04 = 0.14$$

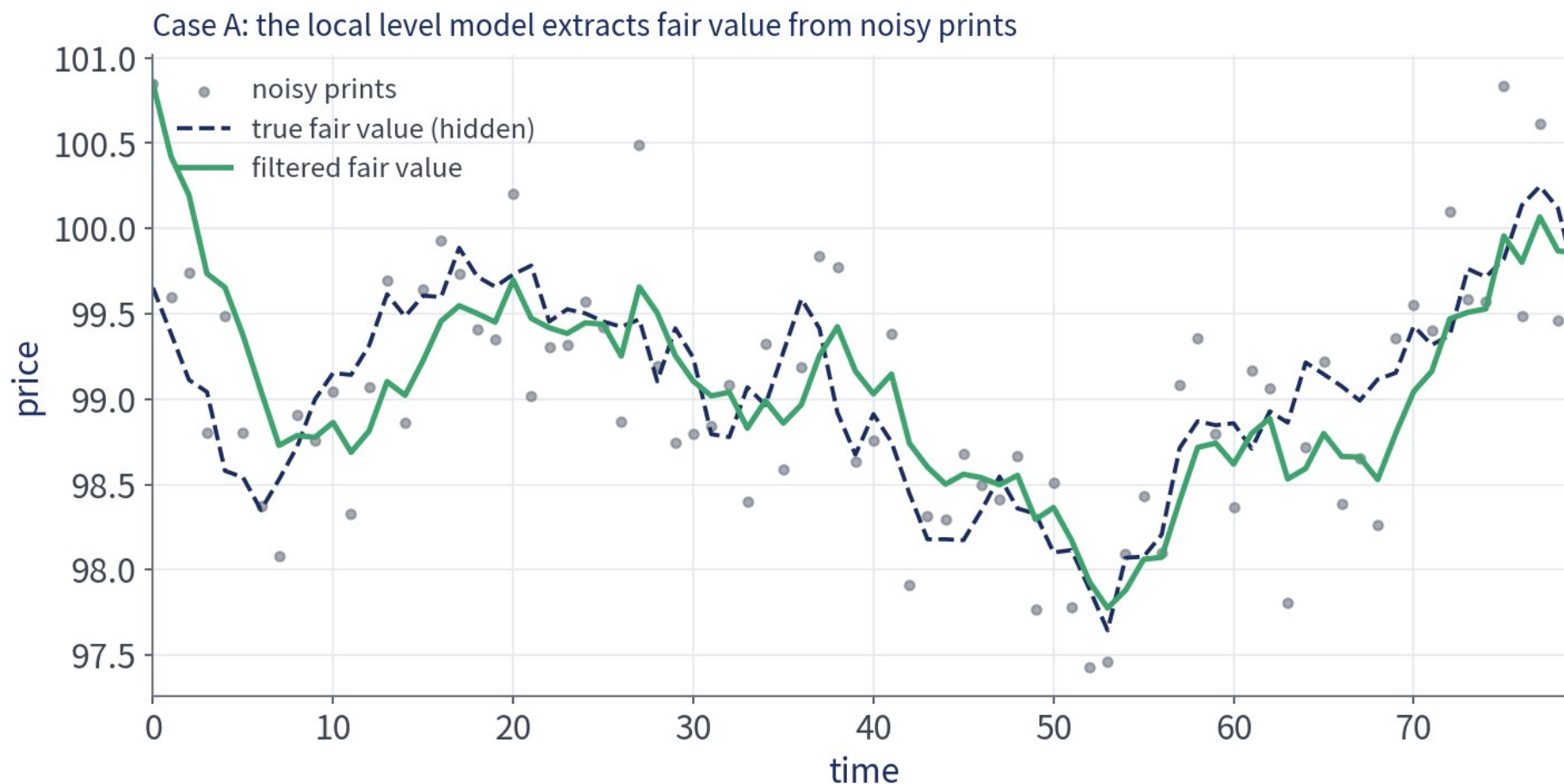
$$\text{gain: } K = \frac{R}{R + \sigma_{\varepsilon}^2} = \frac{0.14}{0.14 + 0.25} = 0.36$$

$$\text{fair value: } 100.00 + 0.36 (100.60 - 100.00) = \mathbf{100.22}$$

The gain is only 0.36, so the 100.60 print pulls our estimate just to 100.22 - we trust the noisy print about a third of the way.

Fair Value, Pulled from Noise

Run that one step over a whole series and the filtered fair value (green) tracks the hidden true value (dashed) while ignoring most of the noise in the scattered prints. The gain decides how closely it follows each print



What the Gain Buys You

A disciplined way to trust a price

Without the model

- You either chase every noisy tick,
- or you pick an arbitrary moving average.
- Neither knows how noisy the data really is.

With the model

- The gain comes from the signal-to-noise ratio.
- Noisier prints are trusted less, automatically.
- You get a fair value, not a guess.

An Index's Level and Its Momentum

The local linear trend model, made concrete

Sometimes the hidden state is moving. An index has a level and a slope - where it is, and how fast it's climbing. The local linear trend model tracks both, and lets the slope itself drift. We predict the next level and update both level and slope from one surprise, by hand

The two hidden pieces

- Level: where the index is now.
- Slope: how fast it's moving (momentum).
- Both are hidden; both can change.

What we'll do by hand

- Predict the next level from level plus slope.
- Measure the surprise against a new print.
- Nudge the level and the slope.

Two Equations for Level and Trend

The slope drifts too

$$\begin{aligned} \text{level: } L_t &= L_{t-1} + S_{t-1} + \text{noise} \\ \text{slope: } S_t &= S_{t-1} + \text{noise} \quad (\text{the drift itself drifts}) \end{aligned}$$

The level

- Next level = level + slope + noise.
- The slope carries it forward each step.
- So momentum is built in.

The slope

- Next slope = slope + noise.
- The drift itself can slowly change.
- That is what 'linear trend' means here.

Predict, Then Update

One surprise moves both level and slope

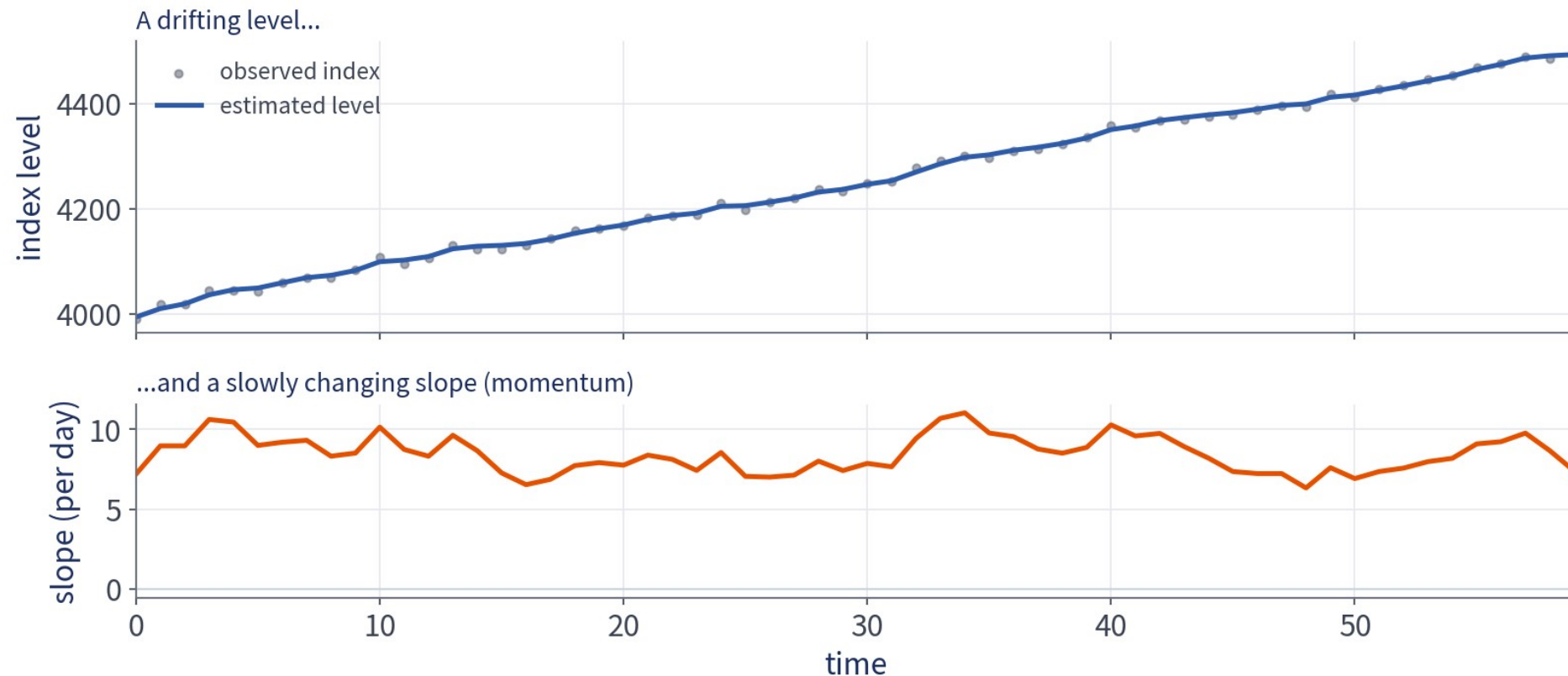
$$\begin{aligned}\text{predict next level} &= L + S = 4000 + 5 = 4005 \\ \text{new obs } 4012 &\Rightarrow \text{surprise} = 4012 - 4005 = +7 \\ \text{level} &\leftarrow 4005 + 0.5(7) = \mathbf{4008.5}, \quad \text{slope} \leftarrow 5 + 0.1(7) = \mathbf{5.7}\end{aligned}$$

The +7 surprise raises the level to 4008.5 and nudges the slope up to 5.7 - the model now believes the index is climbing a little faster.

Level and Slope, Separated

Repeated over a series, the model tracks a smooth level through the noisy index (top) while separately estimating the slope - the momentum - underneath (bottom). Two hidden states, read off one noisy observation

Case B: local linear trend separates the level from the drift



Why Separate Level and Slope

Momentum you can actually measure

What it gives you

- A clean level, free of day-to-day noise.
- A slope estimate - the momentum - as a number.
- Both updated the moment new data arrives.

Why it matters

- A rising slope is a momentum signal.
- A flattening slope warns a trend is fading.
- You separate 'where' from 'how fast'.

The Common Pattern

One template, two hidden states

Both cases follow the same recipe. Decide what you observe and what is hidden, write the two equations, and let the model combine your prior belief with each noisy observation. Fair value or momentum, the machinery is identical - and it is the whole point of state space

The recipe

- Name the observation and the hidden state.
- Write the observation and state equations.
- Combine prior and data by the gain.

Why it's powerful

- The same code fits many finance problems.
- The gain adapts to how noisy the data is.
- Part 2 turns one step into a full filter.

State Space in Finance

The same two-equation template fits problem after problem: fair value from noisy prints, a drifting trend, a time-varying beta, yield-curve factors, stochastic volatility, and nowcasting. Name what's hidden and the filter does the rest

State Space in Finance: What's Hidden?

The same two-equation template - a hidden state, a noisy observation - fits problem after problem.

Fair value

observe: noisy prints / quotes

hidden state:

the true price behind the noise

Trend & momentum

observe: observed index

hidden state:

a drifting level and slope

Time-varying beta

observe: stock vs market returns

hidden state:

a hedge ratio that moves

Term structure

observe: observed bond yields

hidden state:

a few hidden yield factors

Stochastic vol

observe: observed returns

hidden state:

a hidden volatility level

Nowcasting

observe: mixed-frequency data

hidden state:

a hidden state of the economy

One template, many problems

Write down what you observe and what is hidden; the Kalman filter (Part 2) then does the rest.

Reproduce the Hand Calcs, Then Scale Up

Direct Claude Code to check your arithmetic

GOAL Verify both models in code, then run them over a real series.

[INPUT] Prompt · Claude Code

Write and run `part1_practice.py` that:

CASE A - local level:

- * set state var 0.04, obs var 0.25; print q ;
- * from prior 100.00 (var 0.10) and print 100.60, compute the gain and the filtered fair value;
- * then filter a real noisy price series.

CASE B - local linear trend:

- * from level 4000, slope 5, and obs 4012, predict and update level and slope;
- * then track level and slope on a real index.

Match every printed number to the slides.

Check against the slides

- signal-to-noise $q = 0.16$.
- gain $K = 0.36$.
- fair value = 100.22.
- level 4008.5, slope 5.7.

Deliverable: `part1_practice.py` whose printed numbers match the hand calculations, then the charts.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 What decides how much a print moves your estimate?

The gain, from the signal-to-noise ratio

Q2 In the trend model, what does the slope represent?

The momentum - how fast the level moves

Q3 What do the two equations describe?

What you observe, and the hidden state

All three come from the local level filter, the local linear trend update, and the state-space template.

Three Pitfalls to Avoid

Where state-space setups slip

Common mistakes

- "A noisy print is the fair value."
- "The level and the trend are the same thing."
- "Bigger gain is always better."

The reality

- The print is the state plus noise - filter it.
- Level is where; slope is how fast - keep them apart.
- The right gain comes from the signal-to-noise ratio.

Two Calculations to Remember

Part 1 applications in one screen

1

Pull a fair value from a noisy print

With state variance 0.04 and noise variance 0.25, the signal-to-noise ratio is 0.16 and the gain is 0.36 - so a 100.60 print moves the fair value only to 100.22. The model trusts noisy data less, automatically.

2

Separate an index's level from its slope

Predicting level + slope gave 4005; a +7 surprise raised the level to 4008.5 and the slope to 5.7. The local linear trend model tracks where the index is and how fast it's moving, as two numbers.

3

One template, many problems

Name what you observe and what is hidden, write two equations, and combine prior and data by the gain - the same recipe fits fair value, momentum, beta, yields, and volatility.

Next — Part 2 practice: the full Kalman filter and smoother, run over a series by hand and in code.

★ WEEK 8 · PART 2 of 3

Filtering, Smoothing, Forecasting

Three Questions, One Recursive Engine

Real-time estimates · hindsight · projection · a drifting hedge ratio

Given the two equations, three questions follow: what is the state now, what was it then, and what will it be? Each conditions on a different slice of data — and one recursive cycle answers them all.

2

PART 2 · INFERENCE

Estimating the Hidden State

Filtering, smoothing, and forecasting, the predict-update cycle that powers them, and the time-varying hedge ratio that carries the pairs thread into state space.

We Have the Model — Now, the Inference

Part 1 set up the state; Part 2 estimates it

1

The state is hidden

Part 1 gave us the two equations: a state evolving over time, seen only through noisy observations. But we never wrote down HOW to recover the state.

2

Three questions, one framework

What is the state now (filtering)? What was it, in hindsight (smoothing)? What will it be (forecasting)? All three fall out of the same model.

3

One recursive cycle answers them

A predict-then-update loop estimates the state in real time — the engine we will make optimal as the Kalman filter (Week 9).

Part 2 in one line: filter, smooth, and forecast the hidden state — all from one recursive cycle.

Three Questions, One Framework

Same goal, different information sets

Estimating a hidden state comes in three flavors, distinguished only by WHICH data you condition on: everything up to now, the whole sample, or nothing beyond now.

The three tasks

- Filtering: the state NOW, from data up to now.
- Smoothing: the state THEN, using all data in hindsight.
- Forecasting: the state AHEAD, projecting beyond now.

The unifying point

- The model is identical in all three.
- Only the conditioning information changes.
- So one engine, run differently, does all three.

The Three Inference Problems

One line each

$$\underbrace{\mathbb{E}[x_t \mid y_{1:t}]}_{\text{filtering}} \quad \underbrace{\mathbb{E}[x_t \mid y_{1:T}]}_{\text{smoothing}} \quad \underbrace{\mathbb{E}[x_{T+h} \mid y_{1:T}]}_{\text{forecasting}}$$

Reading them

- Filtering conditions on data up to time t .
- Smoothing conditions on the entire sample, up to T .
- Forecasting conditions on data up to T , then projects forward.

Why the distinction matters

- Filtering is what you can do LIVE, as data arrives.
- Smoothing is the best estimate, but only after the fact.
- Forecasting is filtering's natural extension into the future.

Filtering: The State Now

The best real-time estimate

What it is

- The estimate of the state at time t , using data up to t .
- It updates the instant a new observation arrives.
- It is causal — it never peeks at the future.

Why it is central

- It is what you use in live trading and control.
- It powers real-time signals and risk estimates.
- It is the output of the recursive filter.

Smoothing: The State Then

The best estimate, with hindsight

What it is

- The estimate of the state at a past time, using ALL the data.
- It borrows information from AFTER that moment.
- So it is smoother and more accurate than filtering.

When to use it

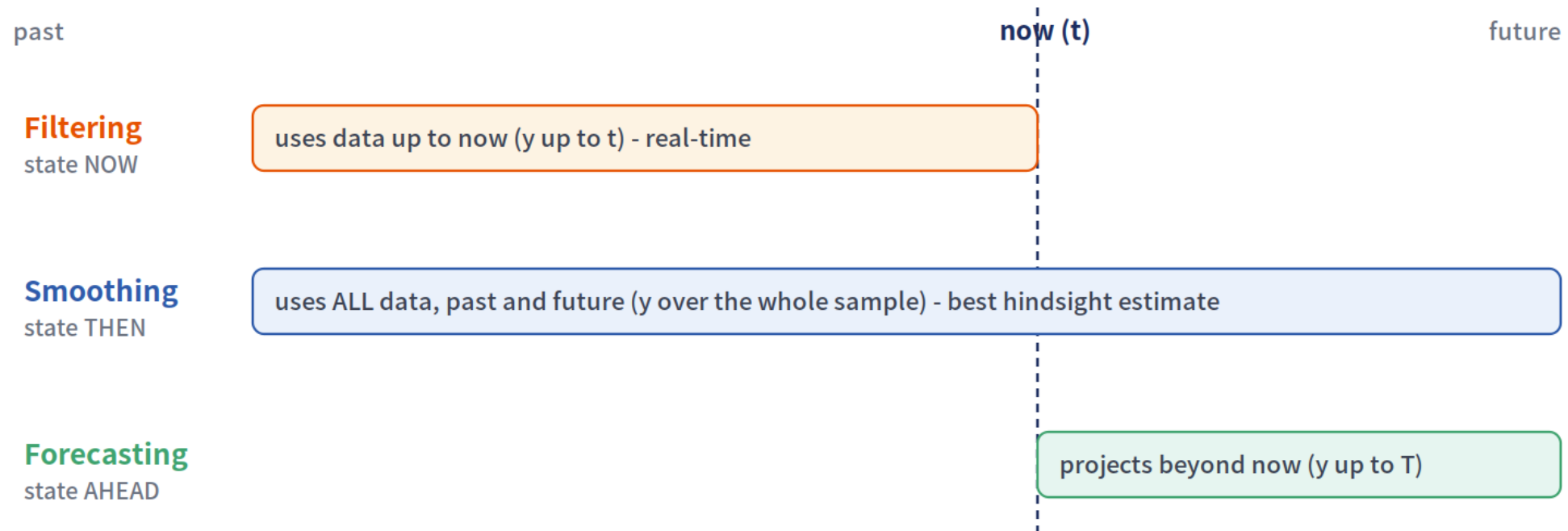
- For analysis and decomposition after the fact.
- To read a clean trend or seasonal component from history.
- Never for real-time decisions — it uses future data.

Which Data Does Each Use?

Filtering conditions on data up to now; smoothing uses the whole sample, past and future; forecasting projects beyond now. Same model, three information sets.

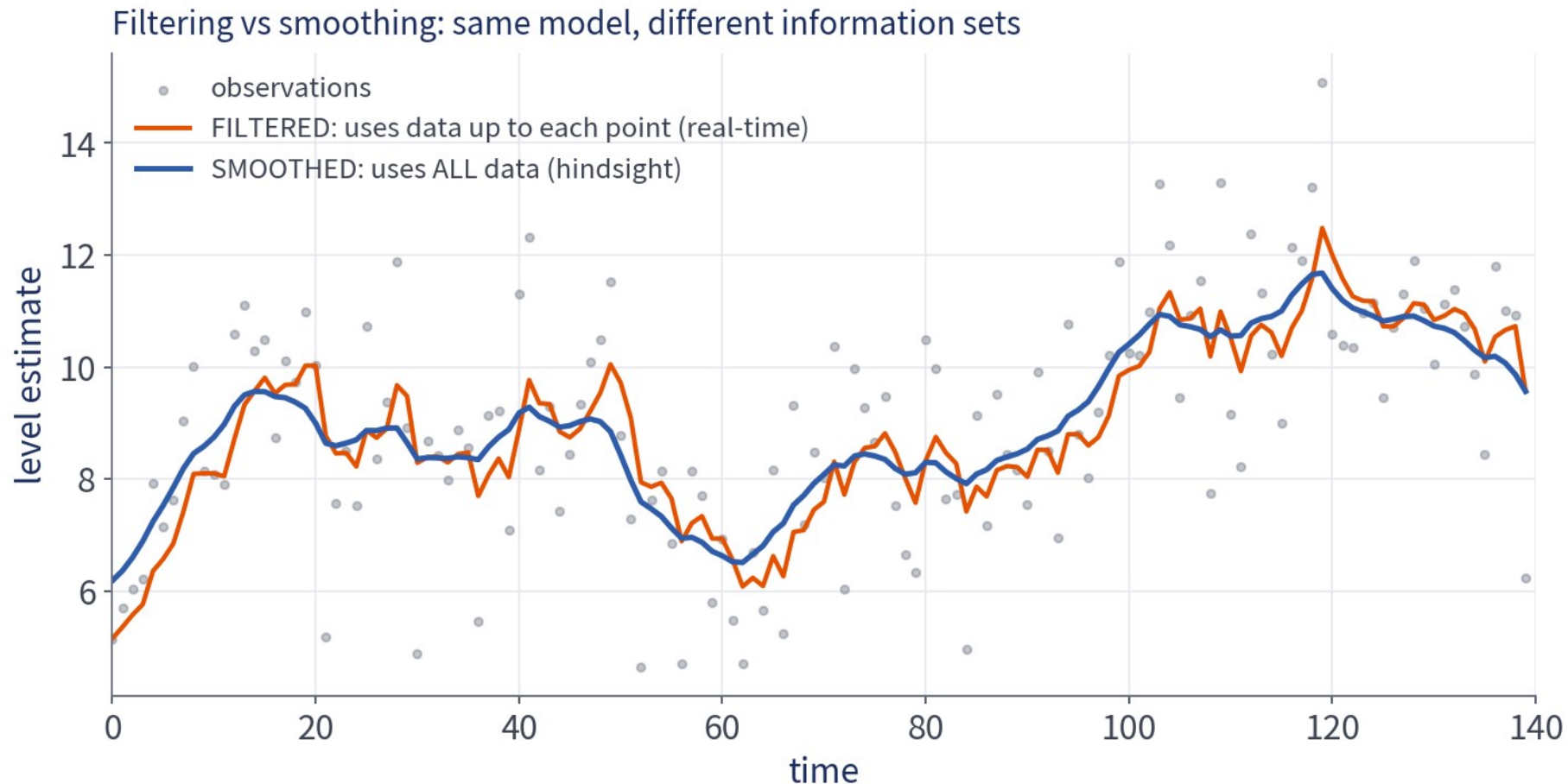
Which Data Does Each Use?

Filtering, smoothing, and forecasting differ only in the information set they condition on.



Filtered vs Smoothed

The filtered estimate (orange) is real-time — it lags and wobbles at the edges. The smoothed estimate (blue) uses all data, so it is cleaner — but only available after the fact.



Filtering vs Smoothing

The same estimate, two information sets

**Filtering answers 'what is the state now, given what I know now?'.
Smoothing answers 'what was the state, given everything I
eventually learned?'. Hindsight makes smoothing cleaner.**

Filtering

- Causal, real-time, usable live.
- Noisier and slower to react at the edges.
- The right tool for trading and monitoring.

Smoothing

- Uses future data — only available in hindsight.
- Cleaner and more accurate for every past point.
- The right tool for analysis and reporting.

Real-Time Updating

Revise the estimate as each point arrives

The recursion

- Carry forward the current best estimate and its uncertainty.
- When new data arrives, correct the estimate.
- No need to refit on the whole history each time.

Why it is efficient

- Each step is a light, constant-cost update.
- It scales to streaming, high-frequency data.
- This recursive structure is the filter's great virtue.

The Predict-Update Cycle

One step of the recursion

$$\text{predict: } x_{t|t-1} = T x_{t-1|t-1} \quad \text{update: } x_{t|t} = x_{t|t-1} + K_t (y_t - Z x_{t|t-1})$$

Predict

- Push the state forward by the transition.
- Its uncertainty grows — you know a little less.
- This gives a forecast of the next observation.

Update

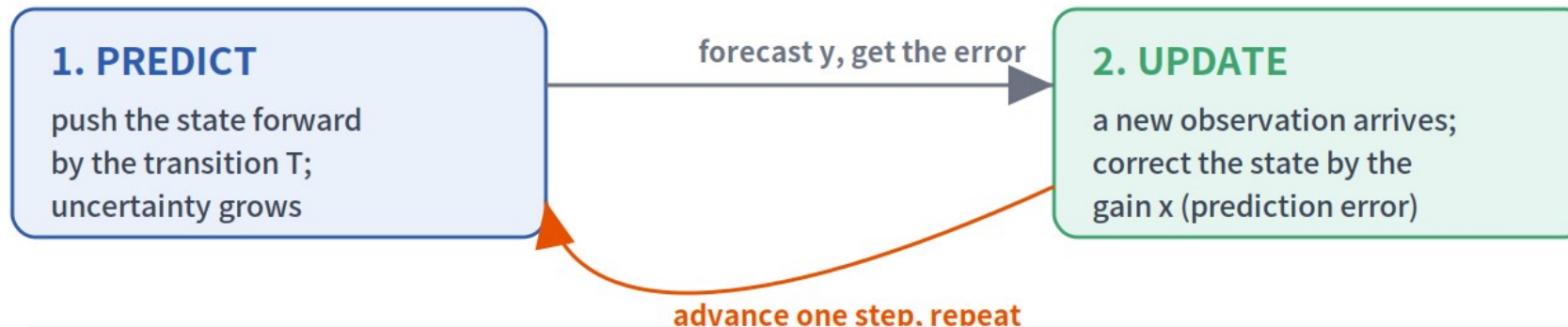
- Compare the actual observation to that forecast.
- Correct the state in proportion to the error, via the gain K .
- The gain decides how much the new point moves you.

The Predict-Update Cycle

Guess ahead, see the new data, correct by the gain — then advance and repeat. The gain K balances trust in the model against trust in the data; the Kalman filter makes it optimal.

The Predict-Update Cycle

Filtering is a loop: guess ahead, see the new data, correct - then repeat.



The gain K is the whole trick

- large gain: the state is uncertain or the data is precise $\rightarrow$ trust the new point
 - small gain: the state is well-pinned or the data is noisy $\rightarrow$ barely move
- This optimal gain is exactly what the Kalman filter computes (Week 9).

Forecasting From a State Space Model

Filtering, continued into the future

To forecast, just keep predicting without updating — there is no new data to correct with. The state coasts forward by its dynamics, and the uncertainty grows each step.

How it works

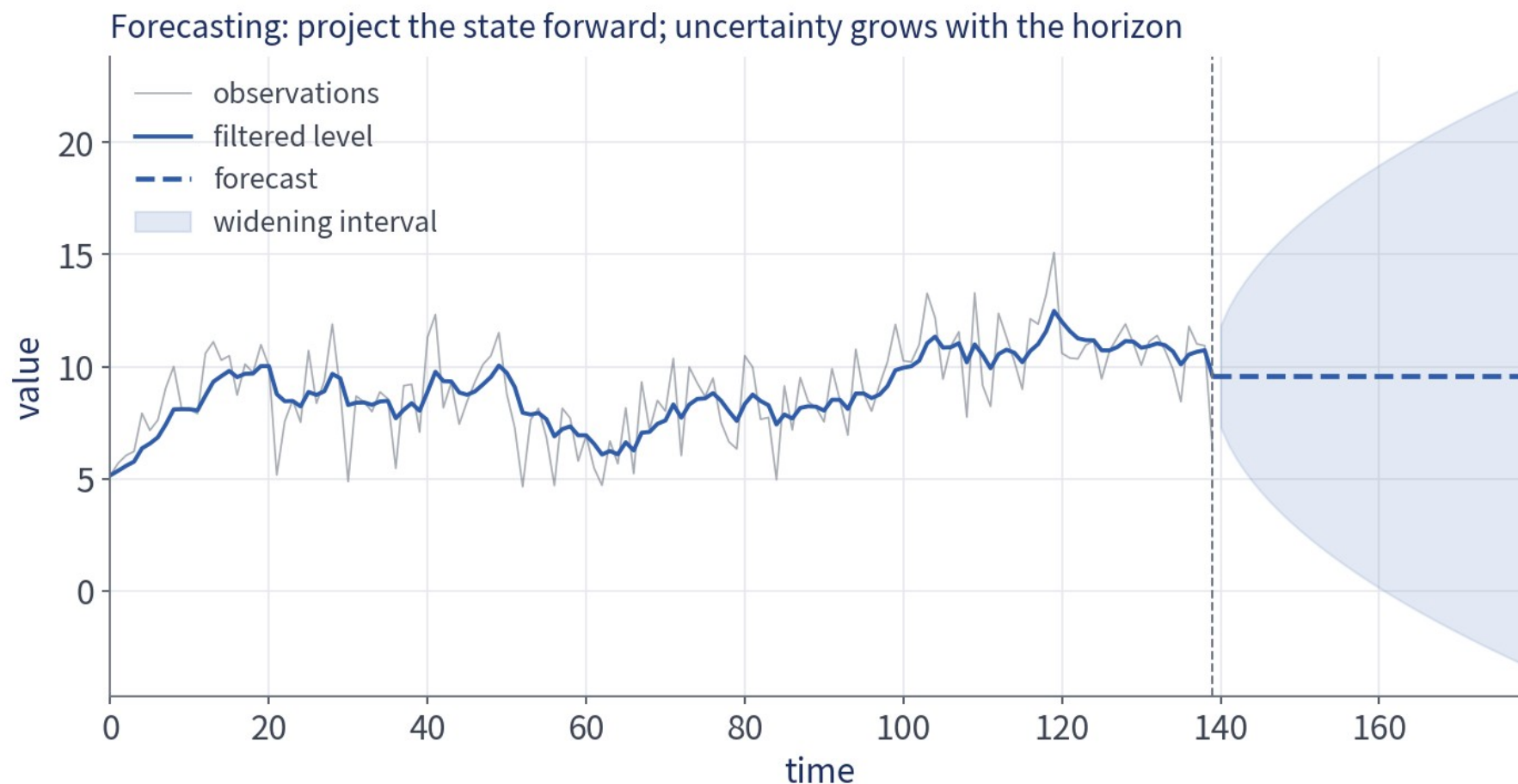
- Run the predict step repeatedly, with no updates.
- The state evolves by the transition alone.
- Map it through the observation equation to forecast the data.

What you get

- A forecast path for both the state and the observations.
- Intervals that widen naturally with the horizon.
- Consistent with the same model that filtered the past.

Forecasting With Growing Uncertainty

Past the last observation, the filter keeps predicting with no data to correct it. The point forecast coasts on the state's dynamics; the interval fans out with the horizon.



Uncertainty, for Free

Error bars come with every estimate

you also get the uncertainty for free: $P_{t|t} = \text{Var}(x_t \mid y_{1:t})$

What you get

- Alongside each state estimate, its variance.
- That gives calibrated error bars at every point.
- The intervals on the forecasts come from it directly.

Why it matters

- You always know how much to trust the estimate.
- The gain uses this uncertainty to weight new data.
- Risk and sizing decisions can use it explicitly.

Time-Varying Parameters, Applied

Let a coefficient be a hidden state

The most valuable trick: make a model's coefficient itself a hidden state that drifts over time. The filter then estimates that coefficient at every moment — a model that adapts.

The setup

- Write the coefficient as a state following a random walk.
- The observation equation uses that drifting coefficient.
- Filtering tracks its value through time.

The finance payoff

- Betas, exposures, and hedge ratios all change with regime.
- A single fixed number is stale the moment things shift.
- State space gives a live, adapting estimate.

The Dynamic Hedge Ratio

The pairs thread enters state space

$$\text{dynamic hedge ratio: } Y_t = \beta_t X_t + \varepsilon_t, \quad \beta_t = \beta_{t-1} + \eta_t$$

The model

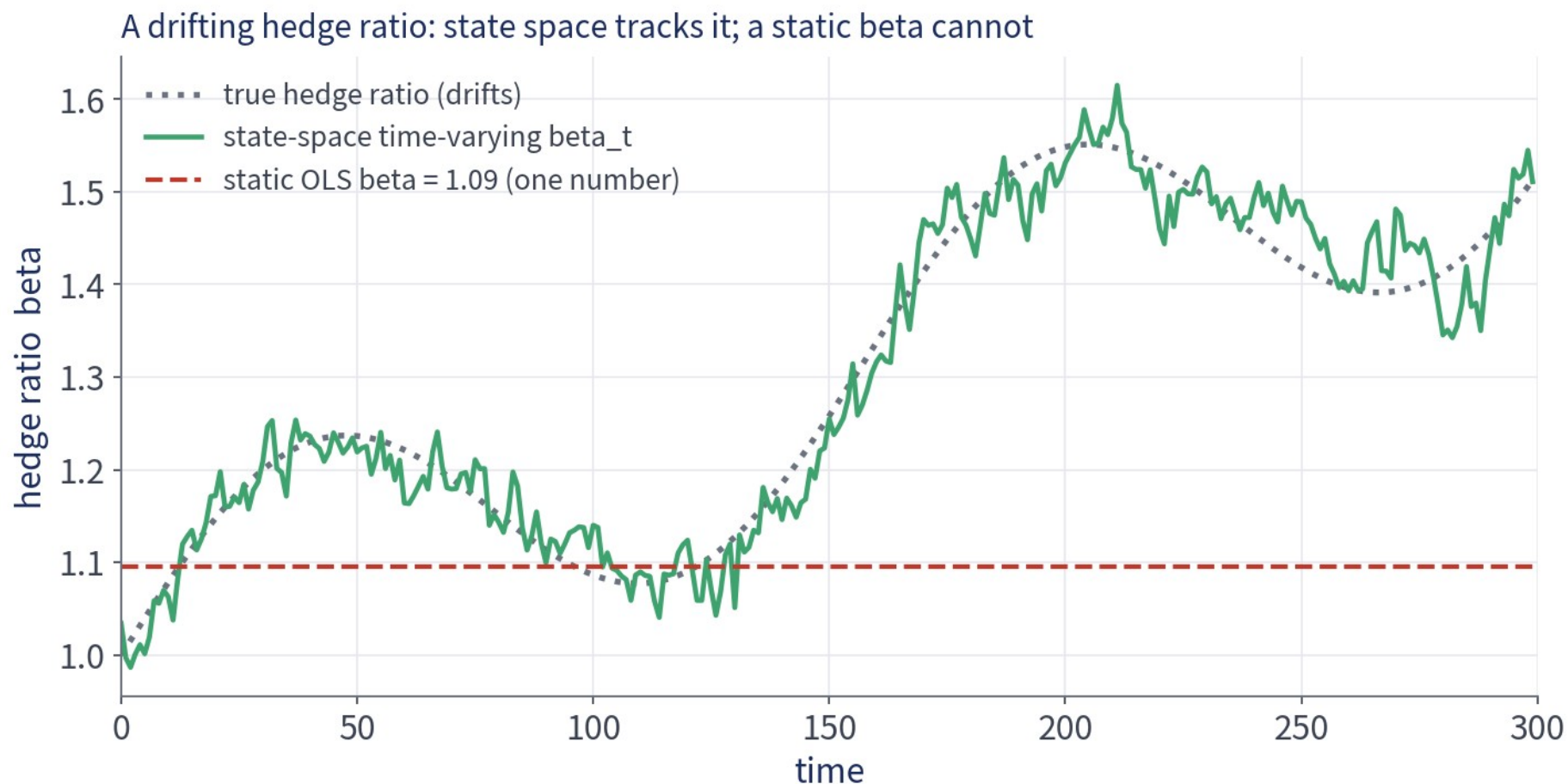
- The hedge ratio beta becomes a drifting hidden state.
- It follows a random walk instead of a fixed value.
- The filter re-estimates it as new prices arrive.

Why it beats a fixed beta

- The true relationship between two assets shifts over time.
- A static hedge ratio (Week 7) slowly goes stale.
- A drifting beta keeps the spread properly hedged.

A Drifting Hedge Ratio

When the true hedge ratio drifts, a single static OLS beta is a poor compromise. The state-space estimate tracks the drift in real time — a self-updating hedge.



Why a Drifting Beta Wins

Adapting beats assuming

The static problem

- One OLS beta averages over the whole history.
- It is wrong at the start and wrong at the end of a drift.
- The 'hedge' leaks exposure whenever the ratio moves.

The dynamic fix

- Track beta as a state; it follows the relationship live.
- The spread stays genuinely market-neutral over time.
- Signals built on it are cleaner and more reliable.

The Pairs-Trading Thread, So Far

Three stages, three weeks

Stage	Week	What it added
Stage I	5	GARCH bands — volatility-scaled thresholds on the spread
Stage II	7	Cointegration — a validated, mean-reverting equilibrium spread
Stage III	8	State space — a drifting, self-updating hedge ratio

Each week deepened the same strategy: better bands, a real anchor, and now an adaptive hedge.

Diagnostics for State Space Models

Checking the fit through the innovations

The key object

- The innovations are the one-step prediction errors.
- In a well-fit model they are standardized white noise.
- They are the state space analogue of residuals.

What to check

- No autocorrelation in the standardized innovations.
- Roughly normal, with constant variance.
- Leftover structure means the state design is incomplete.

Filter, Smooth, and a Dynamic Hedge

Direct Claude Code across all three tasks

GOAL Compare filtering vs smoothing on one series, then estimate a time-varying hedge ratio for a pair.

[INPUT] Prompt · Claude Code

Write and run `filter_smooth_lab.py` that:

1. fits a local level model and plots the FILTERED and SMOOTHED states together, so the difference is visible;
2. produces a multi-step forecast with widening confidence bands;
3. for a pair, fits a time-varying regression (random-walk beta) and plots `beta_t` against the static OLS beta;
4. checks the standardized innovations for autocorrelation.

Say when the dynamic beta diverges from OLS.

How to work with it

- Smoothed should be cleaner than filtered.
- Forecast bands should fan out with the horizon.
- The dynamic beta should drift where OLS is flat.
- Clean innovations = a well-specified model.

Deliverable: `filter_smooth_lab.py` + filter/smooth, a forecast, and a drifting hedge ratio.

Three Questions Before You Move On

If you can answer these, Part 2 has landed

Q1

You need a state estimate for live trading. Filtering or smoothing?

Filtering (causal)

Q2

What does the gain K balance in the update step?

Model vs data trust

Q3

Why model a hedge ratio as a drifting state instead of one OLS number?

The relationship changes

All three come from the information sets, the predict-update cycle, and time-varying parameters.

Three Misconceptions to Avoid

Where inference intuition slips

Common myths

- "Smoothing is just a better filter — use it live."
- "Forecasting needs a different model from filtering."
- "A time-varying beta is over-fitting."

The reality

- Smoothing uses FUTURE data — never available in real time.
- Forecasting is the same filter, predicting with no updates.
- A drifting beta with tuned noise adapts without chasing noise.

Three Things to Carry Forward

Part 2 in one screen

1

Filter, smooth, forecast — one model

They differ only in the data they condition on: up to now (filter), the whole sample (smooth), or projected ahead (forecast). One engine serves all three.

2

A predict-update cycle does it, with uncertainty

Guess ahead, then correct by the gain when data arrives. Each estimate comes with calibrated error bars — the gain uses that uncertainty to weight new data.

3

Time-varying parameters carry the pairs thread

Make the hedge ratio a drifting state, and the filter tracks it live. That is Stage III of pairs trading — an adaptive, self-updating hedge.

Next — Part 3: the midterm synthesis, tying Weeks 1-7 into one connected story.

★ WEEK 8 · PART 2 · PRACTICE

The Kalman Filter, Worked by Hand

Run the Full Recursion; Track a Drifting Hedge Ratio

Predict and update, step by step, on real financial problems

Part 2's engine, with the arithmetic shown. We run the predict-update recursion over four steps and watch the gain settle, then track a time-varying hedge ratio - the pairs thread - with one Kalman update by hand.

2P

PART 2 · APPLICATIONS

The Recursion, Computed by Hand

Case A: run the Kalman recursion over four prints, watch the gain reach a steady state, and see smoothing beat filtering. Case B: track a dynamic hedge ratio with one update. Then a Claude Code lab.

What Part 2 Told Us

Now we run it - with numbers

1

Three inference problems

Filtering estimates the state now, smoothing the state then with hindsight, forecasting the state ahead.

2

The predict-update cycle

Predict the state forward, then correct it with each new observation by the Kalman gain. We run four steps by hand.

3

Parameters can be hidden states

A hedge ratio that drifts is just another state to filter. We update one by hand - the pairs thread in state space.

Everything here is a calculator exercise: four prints, one recursion, one return pair.

How These Cases Work

Data, then arithmetic, then meaning

Each case has three steps

- Start from the recursion or the two equations.
- Run predict-update by hand, step by step.
- Then read what the filtered state tells you.

What you should watch

- How the gain settles to a steady state.
- How smoothing improves on filtering.
- How a hedge ratio drifts and gets tracked.

Run the Full Recursion

From one step to a real-time filter

Part 1 did one filtering step; now we run the recursion. Using the same local level model - a fair value hidden behind noisy prints - we predict and update across four prints, tracking the estimate and its variance, and watch the Kalman gain settle to a steady state.

The recursion

- Predict the state and its variance forward.
- Correct it with each print by the gain.
- Carry the variance to the next step.

What we'll do by hand

- Run four predict-update steps.
- Watch the gain converge near 0.33.
- See the estimate track the fair value.

The Predict-Update Cycle

Four formulas, run every step

$$\begin{aligned}\text{predict: } R &= C + \sigma_{\eta}^2 \\ \text{gain: } K &= \frac{R}{R + \sigma_{\varepsilon}^2} \\ \text{update mean: } m &\leftarrow m + K (P - m) \\ \text{update var: } C &\leftarrow (1 - K) R\end{aligned}$$

Each step: grow the variance (predict), form the gain, correct the mean toward the print, and shrink the variance (update). Then repeat.

Four Steps of the Recursion

Predict and update, print by print

t	print P	R (predict)	gain K	m (filtered)	C (var)
1	100.6	0.140	0.359	100.215	0.090
2	100.2	0.130	0.342	100.210	0.085
3	100.9	0.125	0.334	100.440	0.084
4	100.5	0.124	0.331	100.460	0.083

The gain falls from 0.359 toward a steady 0.33, and the variance settles - the filter reaches an equilibrium as data accumulates.

One Full Step, in Detail

Step 3, worked end to end

$$\begin{aligned}\text{step 3: } R &= 0.0854 + 0.04 = 0.125, & K &= \frac{0.125}{0.125 + 0.25} = 0.334 \\ m &= 100.21 + 0.334(100.9 - 100.21) = \mathbf{100.44}, & C &= (1 - 0.334) 0.125 = 0.084\end{aligned}$$

What happened

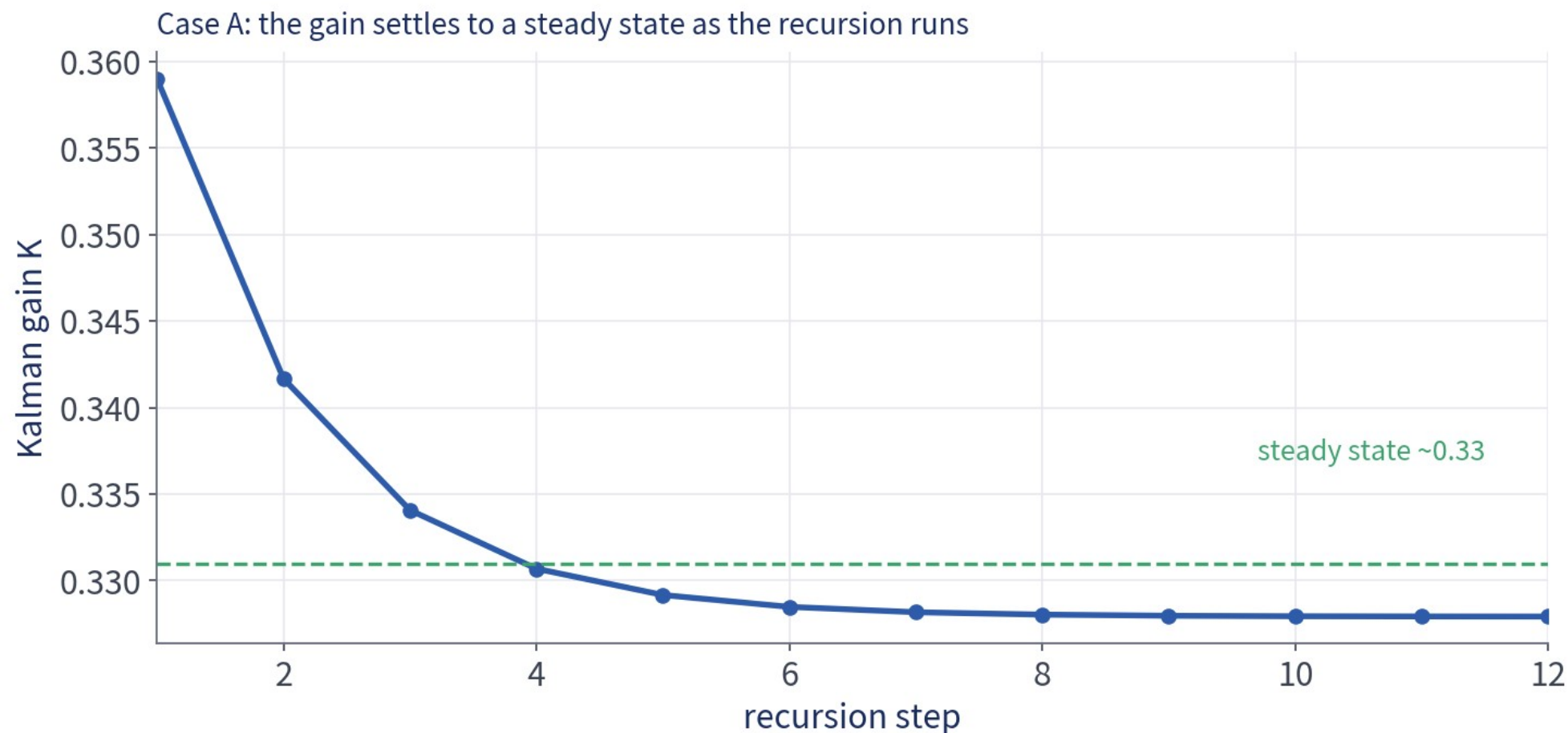
- The print 100.9 was above the estimate.
- The gain 0.334 pulled the mean up to 100.44.
- The variance shrank to 0.084.

Why it settles

- Predict grows the variance by 0.04.
- Update shrinks it by the gain.
- The two balance - a steady-state gain.

The Gain Settles

Run the recursion longer and the Kalman gain converges to a steady state near 0.33. After that, every print is trusted by the same fixed fraction - the filter has learned the right balance of signal and noise for this model



Filtering vs Smoothing

Real time, or with hindsight

Filtering (real time)

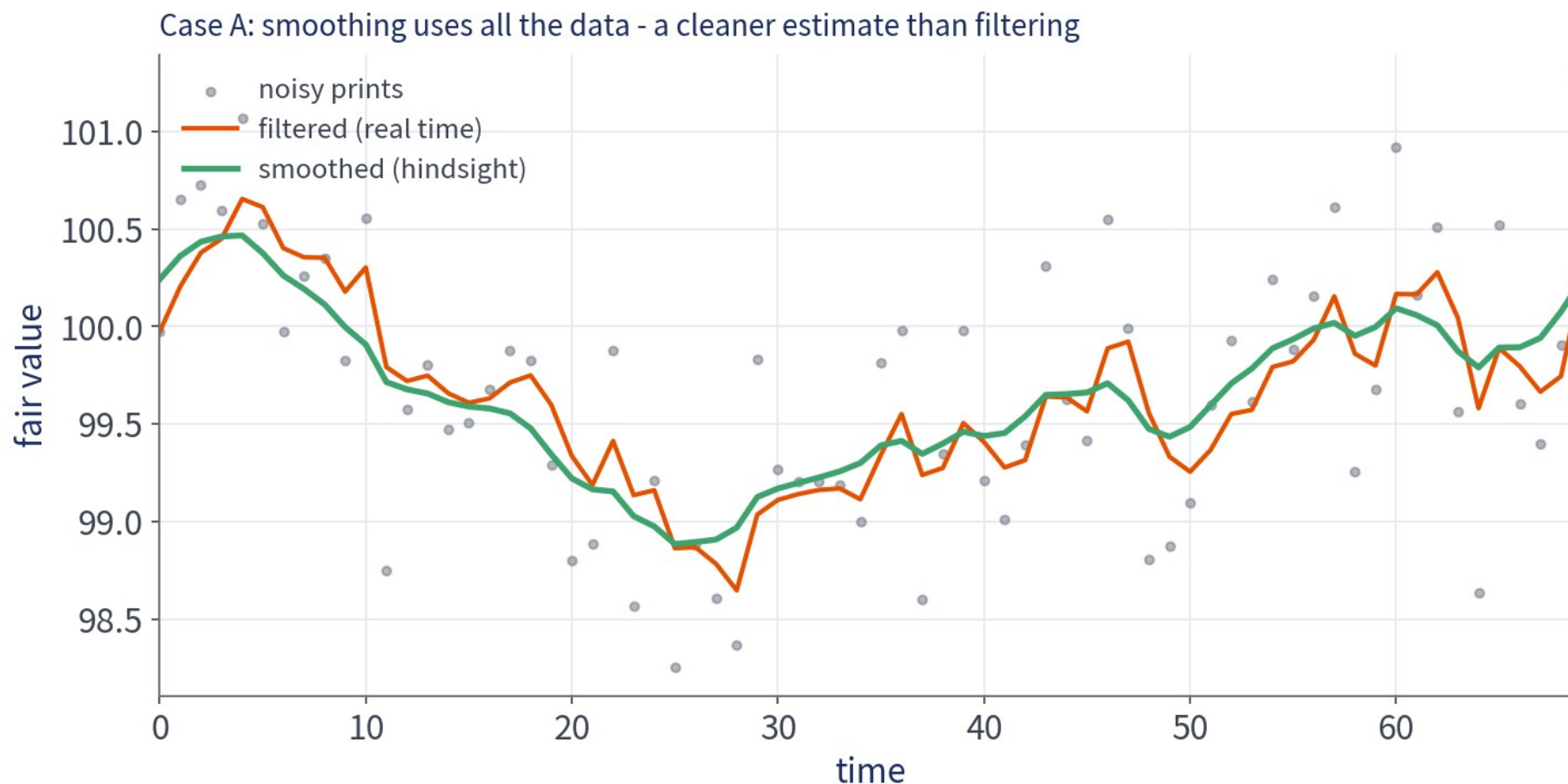
- Uses only data up to now.
- The best you can do live, at each step.
- A little jumpy - it can't see ahead.

Smoothing (hindsight)

- Uses the whole series to revise each point.
- A cleaner estimate of what the state was.
- Only available after the fact - for research.

Smoothing Beats Filtering, After the Fact

The filtered estimate (orange) uses only the past, the smoothed estimate (green) uses the whole series to revise each point, so it is cleaner and less jumpy. Use filtering to trade live, smoothing to study what really happened



What the Recursion Buys You

A living estimate, not a static average

The filter gives you

- A fair value updated every tick.
- A variance - how sure you are right now.
- A gain that adapts to the noise level.

The choice of tool

- Filter when you must act in real time.
- Smooth when you're analyzing history.
- Same model, two information sets.

A Hedge Ratio That Drifts

The pairs thread enters state space

In pairs trading you hedge stock A with a multiple of stock B - the hedge ratio. A fixed ratio from one regression goes stale. Treat the hedge ratio as a hidden state that drifts, and the Kalman filter updates it as new returns arrive. We do one update by hand.

The setup

- Regress A's return on B's return.
- The slope is the hedge ratio, beta.
- Let beta drift slowly over time.

What we'll do by hand

- Write beta as a hidden state.
- Take one new return pair.
- Update beta with one Kalman step.

The Hedge Ratio as a Hidden State

A drifting regression slope

$$\begin{aligned} \text{observation: } r_{A,t} &= \beta_t r_{B,t} + \varepsilon_t && \text{(regress A on B)} \\ \text{state: } \beta_t &= \beta_{t-1} + \eta_t && \text{(the hedge ratio drifts)} \end{aligned}$$

The observation

- A's return equals beta times B's, plus noise.
- The regressor is B's return - it changes daily.
- So the gain changes each step too.

The state

- Beta takes a small random step each day.
- It drifts, but slowly.
- That is the hidden thing we track.

One Update of the Hedge Ratio

From a single return pair

$$\text{predict var: } P = 0.040 + 0.001 = 0.041$$

$$\text{innovation: } e = r_A - \beta r_B = 1.2 - 0.80(1.0) = 0.40$$

$$\text{gain: } K = \frac{P r_B}{r_B^2 P + \sigma_\varepsilon^2} = \frac{0.041}{0.041 + 0.20} = 0.17$$

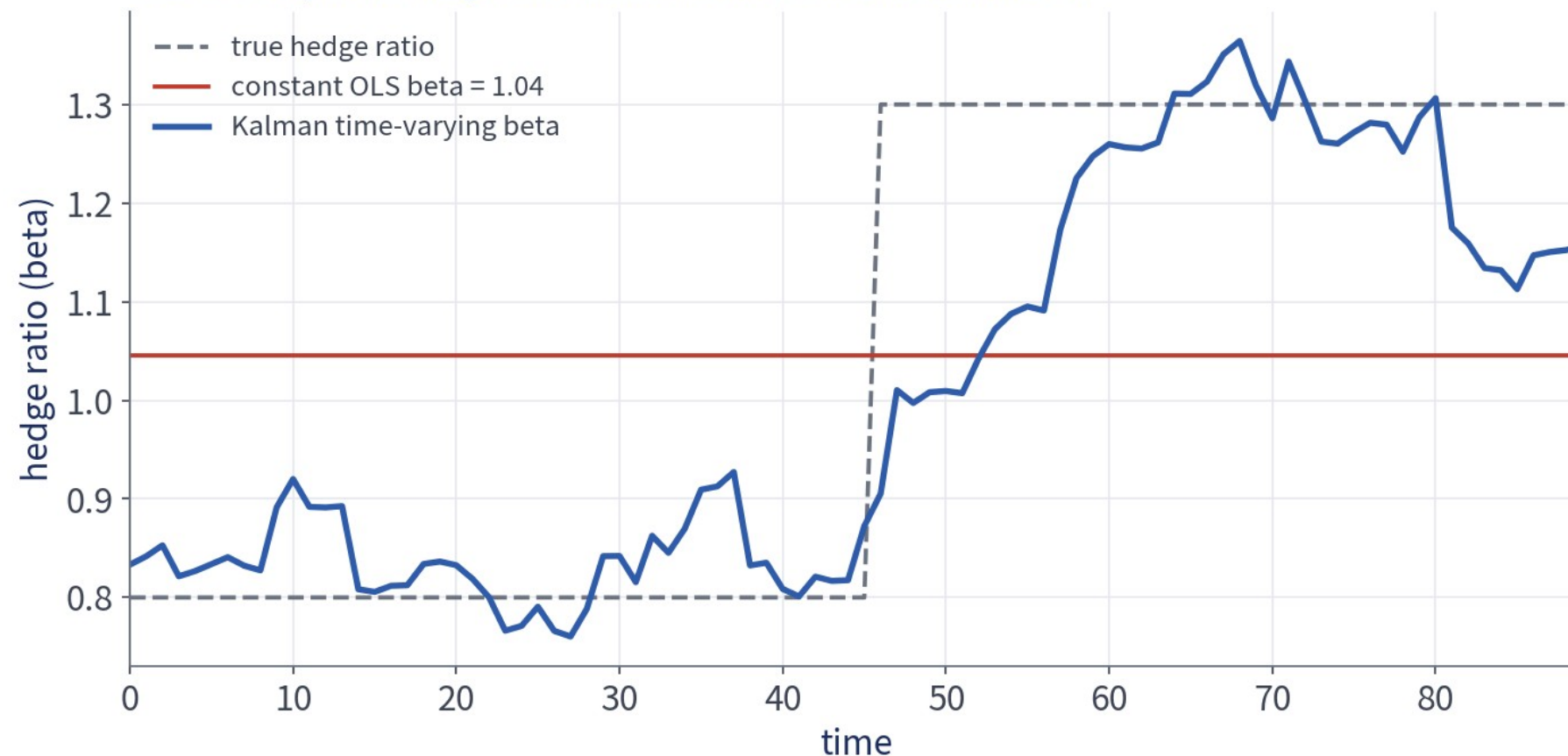
$$\text{update: } \beta \leftarrow 0.80 + 0.17(0.40) = \mathbf{0.87}$$

A's 1.2% return beat 0.80 x B's 1.0%, an innovation of 0.40. The gain 0.17 nudges beta from 0.80 up to 0.87 - a gradual, sensible revision.

The Dynamic Hedge Ratio Tracks the Shift

When the true hedge ratio jumps, the Kalman estimate (blue) follows it within a few steps, while a single constant OLS beta (red) sits stuck in between - hedging badly both before and after. A drifting state beats a fixed number

Case B: the dynamic hedge ratio tracks the shift; constant OLS misses it



Why a Dynamic Hedge Wins

The pairs trade, kept in tune

A constant hedge

- Comes from one regression on old data.
- Goes stale when the relationship shifts.
- Leaves you over- or under-hedged.

A filtered hedge

- Updates the ratio as new returns arrive.
- Adapts to a regime change automatically.
- Keeps the spread properly market-neutral.

The Common Pattern

One recursion, many states

Both cases run the same engine. Whatever the hidden state - a fair value or a hedge ratio - the filter predicts it forward, then corrects it by the gain when data arrives. The gain does the thinking: it weighs your prior against the new observation, automatically.

The recipe

- Predict the state and its variance.
- Form the gain from those variances.
- Correct the state, shrink the variance.

Why it's powerful

- The same loop tracks any hidden state.
- The gain adapts to the noise, step by step.
- Filtering is live; smoothing is hindsight.

Reproduce the Recursion, Then Scale Up

Direct Claude Code to check your arithmetic

GOAL Verify the recursion and the hedge-ratio update, then run both on real data.

[INPUT] Prompt · Claude Code

Write and run `part2_practice.py` that:

CASE A - the recursion:

- * run the predict-update loop over the four prints;
- * print R , K , m , C at each step; show K settling;
- * add an RTS smoother and compare to the filter.

CASE B - dynamic hedge ratio:

- * from beta 0.80 (var 0.04) and returns $r_B=1.0$, $r_A=1.2$, compute the innovation, gain, and new beta;
- * then track beta on a real pair vs a constant OLS.

Match every printed number to the slides.

Check against the slides

- gain settles near 0.33.
- step 3: $m = 100.44$.
- innovation $e = 0.40$.
- beta: 0.80 $\rightarrow$ 0.87.

Deliverable: `part2_practice.py` whose printed numbers match the recursion and the hedge-ratio update.

Three Questions on the Cases

If you can answer these, the practice has landed

Q1 What are the two halves of each recursion step?

**Predict, then update
by the gain**

Q2 How does smoothing beat filtering?

**It uses the whole
series, with hindsight**

Q3 Why filter the hedge ratio instead of fixing it?

**It drifts - a filtered
beta adapts**

All three come from the predict-update cycle, the filter-vs-smoother chart, and the dynamic hedge ratio.

Three Pitfalls to Avoid

Where the recursion slips

Common mistakes

- "Forget to carry the variance forward."
- "Use smoothing to trade in real time."
- "Fit the hedge ratio once and leave it."

The reality

- The variance is the memory - carry it every step.
- Smoothing peeks ahead; only filter live.
- Let the hedge ratio drift and be re-filtered.

Two Calculations to Remember

Part 2 applications in one screen

1

Run the recursion, watch the gain settle

Over four prints the gain fell from 0.359 toward a steady 0.33 and the variance settled. Predict grows the variance, update shrinks it, and the two balance into a steady-state filter.

2

Smoothing beats filtering, after the fact

Filtering uses only the past and is the best you can do live; smoothing uses the whole series to revise each point and is cleaner. Filter to trade, smooth to study.

3

Track a drifting hedge ratio

Treating beta as a hidden state, one return pair ($r_B=1.0$, $r_A=1.2$) nudged it from 0.80 to 0.87 by a gain of 0.17. A filtered hedge adapts to regime shifts a constant OLS beta misses.

Next — Part 3 practice: the midterm synthesis, tying filtering and smoothing into the pairs strategy.

★ WEEK 8 · PART 3 of 3

Midterm Synthesis

Seven Weeks, One Connected Story

The Part I thread · a model decision tree · sample questions · exam tips

Part I was not seven disconnected topics — it was one argument. This synthesis retraces that thread, organizes the toolbox, and prepares you for the midterm on Weeks 1-7.

3

PART 3 · MIDTERM SYNTHESIS

Tying Part I Together

The connected story through Weeks 1-7, a decision tree for choosing a model, a review checklist, worked sample questions, and how to prepare for the exam.

What This Synthesis Is For

Review with understanding, not memorization

1

Part I was one argument

From a single return to a traded spread, each week answered a question the previous one raised. The midterm tests whether you see that thread.

2

Know the concept AND the trigger

Exams ask 'which tool for THIS data?'. So for every model, know what it does and the data pattern that calls for it.

3

This deck retraces and organizes

A roadmap, a decision tree, a checklist, and worked questions — everything you need to consolidate Weeks 1-7.

Part 3 in one line: understand the connected story, and you can answer anything on Weeks 1-7.

One Connected Story

How the seven weeks fit together

Model the MEAN and find it nearly unpredictable; so model the VARIANCE; then model series JOINTLY; and finally their LONG-RUN equilibrium. Each step follows from the limits of the last.

The logical chain

- W1-4: model the mean — returns barely have one.
- W5: so model volatility, where the structure lives.
- W6-7: move to many series, then their equilibrium.

Why it matters for the exam

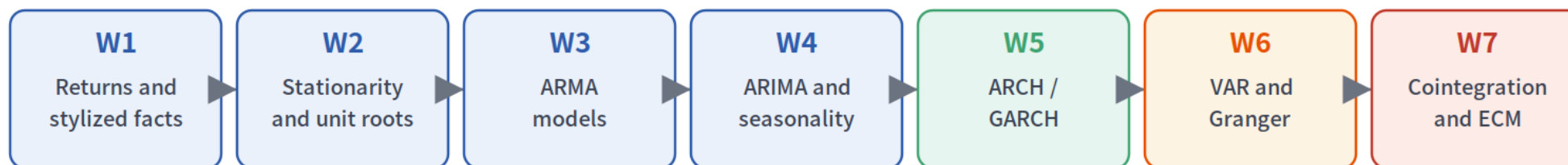
- Questions test transitions, not just definitions.
- 'Why move from ARIMA to GARCH?' is a classic.
- The thread IS the answer key.

Part I: One Connected Story

Model the mean, discover it's unpredictable, model the variance, go multivariate, then capture the long-run equilibrium — with the pairs trade threading through it all.

Part I: One Connected Story

Each week answered a question the previous one raised - from a single return to a traded spread.



The logic that connects them

Model the MEAN (W1-4) - find it is nearly unpredictable - so model the VARIANCE (W5) - then model series JOINTLY (W6) - and finally their LONG-RUN equilibrium (W7). Part II now unifies all of this in state space.

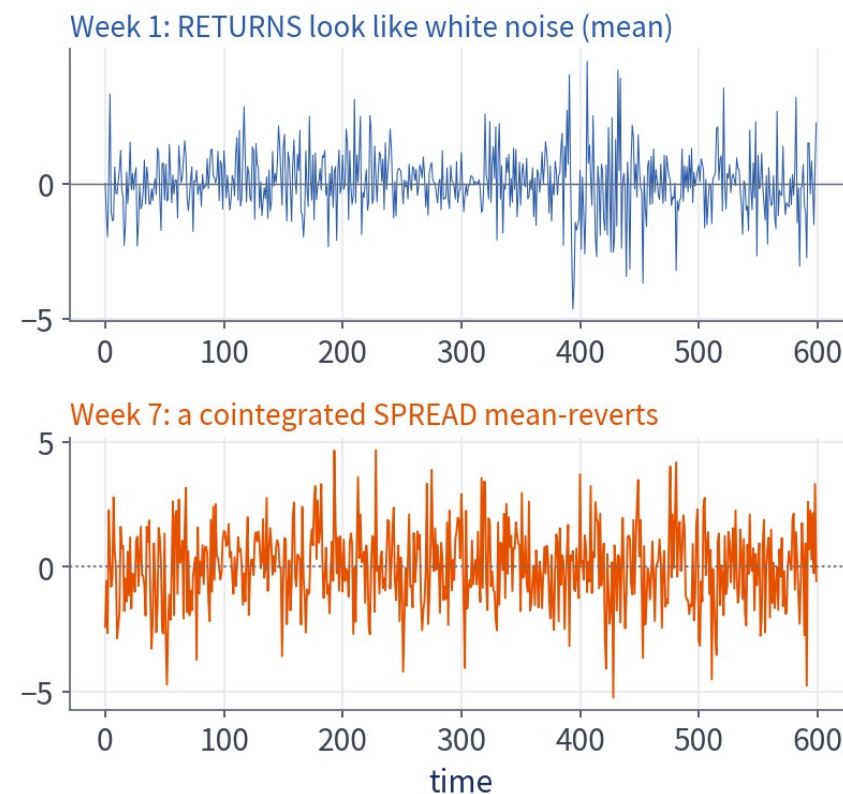
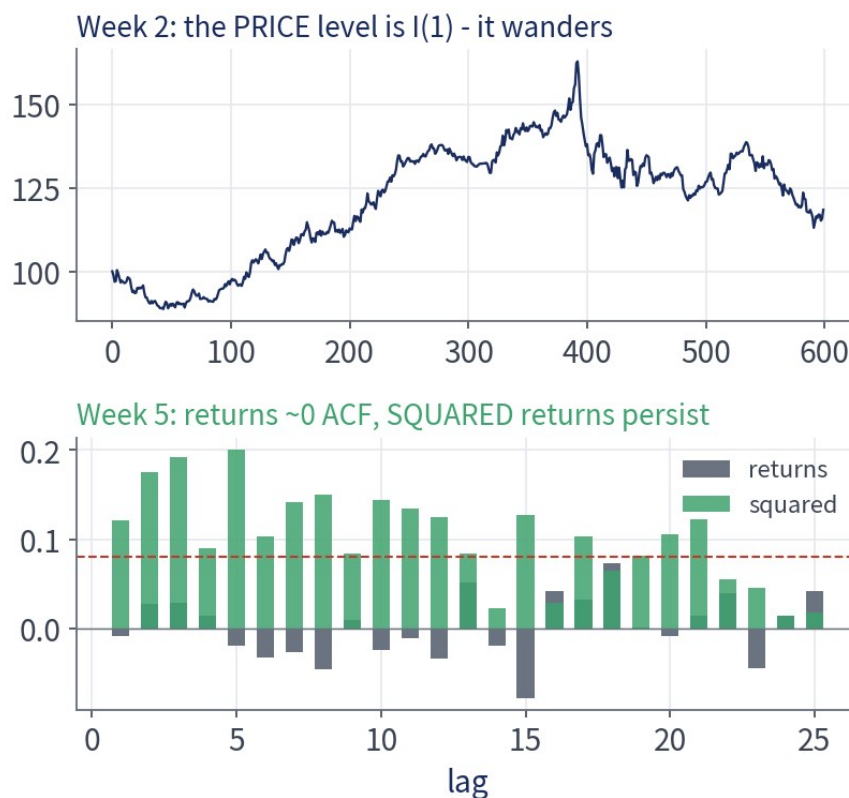
The pairs-trading thread ties it together

Stationarity (W2) - volatility bands (W5) - cointegration anchor (W7) - and, in Part II, an adaptive hedge ratio. If you can trace the pair through all seven weeks, you understand Part I.

One Dataset, Seven Lenses

The same data reads differently through each week's tool: a wandering price (W2), noise-like returns (W1), clustering in squared returns (W5), and a reverting spread (W7).

One dataset, seven lenses: each week of Part I reads the same data differently



Weeks 1-2: Returns and Stationarity

The foundation everything rests on

Key ideas

- Returns, not prices; the stylized facts (fat tails, clustering).
- Stationarity: constant mean and variance over time.
- Unit roots and the $I(0)$ vs $I(1)$ distinction (ADF/KPSS).

When it's tested

- Any question starts here: is the series stationary?
- Spurious regression: high R-squared between $I(1)$ series.
- The answer routes every later modeling choice.

Weeks 3-4: ARMA and ARIMA

Modeling the mean

Key ideas

- ARMA for stationary series; ACF/PACF to identify p and q .
- ARIMA differences d times first; random walk = ARIMA(0,1,0).
- SARIMA adds seasonal terms at lag s (the airline model).

When it's tested

- Identify orders from an ACF/PACF description.
- Choose d from unit-root behavior.
- The humbling fact: returns have almost no ARMA structure.

Week 5: ARCH and GARCH

Modeling the variance

Key ideas

- Conditional vs unconditional variance; volatility clustering.
- GARCH(1,1) with persistence $\alpha + \beta$; mean reversion.
- Student-t for fat tails; GJR/EGARCH for leverage; VaR.

When it's tested

- Flat return ACF but persistent SQUARED-return ACF $\rightarrow$ GARCH.
- Asymmetry after drops $\rightarrow$ GJR/EGARCH.
- Turning a volatility forecast into VaR.

Week 6: VAR and Granger

Modeling series jointly

Key ideas

- VAR: each series on the past of all series; stability, lag order.
- Granger causality tests PREDICTION, not true causation.
- IRF traces shocks; FEVD budgets variance (needs orthogonalization).

When it's tested

- 'Does X Granger-cause Y' — and the causality caveat.
- Reading an IRF or FEVD plot.
- Why ordering matters for Cholesky-based analysis.

Week 7: Cointegration and ECM

The long-run equilibrium

Key ideas

- Two $I(1)$ series with a stationary combination — cointegrated.
- Test with Engle-Granger or Johansen (the rank).
- Error correction: the VECM, speed of adjustment, half-life.

When it's tested

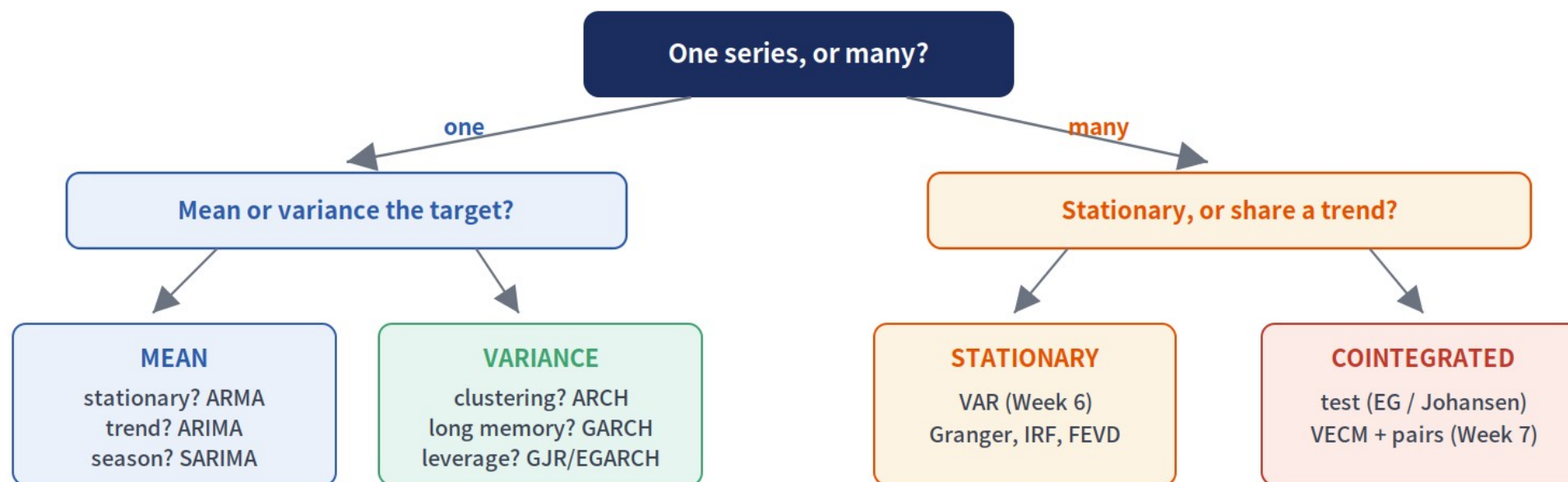
- Spurious vs cointegrated: is the residual stationary?
- Reading alpha (adjustment speed) and the half-life.
- Pairs trading built on the spread — and its breakdown risk.

Which Model, When?

One series or many? Mean or variance? Stationary or sharing a trend? The tree routes any dataset to the right tool — and it always begins with a stationarity check.

Which Model, When?

A decision tree over the Part I toolbox.



Always start by checking stationarity (Week 2) - it routes the entire decision.

The Toolbox on Two Axes

What you model, and how many series

Target	Single series	Multiple series
Mean / level	ARMA, ARIMA, SARIMA	VAR
Variance / risk	ARCH, GARCH, GJR/EGARCH	(multivariate GARCH — later)
Long-run link	unit roots, differencing	cointegration, VECM

Almost every Part I model lives in one of these cells — locate the data, and you locate the tool.

Key Equations: Mean Models

The ones worth memorizing

ARMA(p, q) : $\phi(L)X_t = \theta(L)\varepsilon_t$ stationary if roots of ϕ outside unit circle

ARIMA(p, d, q) : $\phi(L)(1 - L)^d X_t = \theta(L)\varepsilon_t$ RW = ARIMA(0, 1, 0)

SARIMA(p, d, q)(P, D, Q) $_s$: add seasonal terms at lag s

What to know

- The lag-operator form, and what p, d, q each control.
- Stationarity: AR roots outside the unit circle.
- Random walk = ARIMA(0,1,0); seasonal terms at lag s .

How it is tested

- Map an ACF/PACF pattern to orders.
- State the differencing needed for a trend.
- Recognize special cases by name.

Key Equations: Risk and Multivariate

The rest of the must-knows

$$\begin{aligned}\text{GARCH}(1,1) : \sigma_t^2 &= \omega + \alpha \varepsilon_{t-1}^2 + \beta \sigma_{t-1}^2 && \text{persistence} = \alpha + \beta \\ \text{VAR}(p) : \mathbf{Y}_t &= \mathbf{c} + A_1 \mathbf{Y}_{t-1} + \cdots + A_p \mathbf{Y}_{t-p} + \boldsymbol{\varepsilon}_t \\ \text{Cointegration} : X_t, Y_t &\sim I(1), \quad z_t = Y_t - \beta X_t \sim I(0)\end{aligned}$$

What to know

- GARCH(1,1) and persistence alpha + beta.
- The VAR form and its cross-terms.
- The cointegration condition: a stationary spread of I(1) series.

How it is tested

- Interpret alpha + beta near 1 (persistent volatility).
- Read off-diagonal VAR terms as lead-lag.
- Identify cointegration from a stationary residual.

The Pairs-Trading Thread

One example that touches every week

If you can trace the pairs trade through Part I, you understand the whole. Stationarity finds the pair; volatility sizes the bands; cointegration anchors it; error correction is why it reverts.

Week by week

- W2: are the two series $I(1)$? (the prerequisite)
- W5: GARCH bands that scale with volatility.
- W7: cointegration test, spread, and error-correction speed.

The exam angle

- A single pairs scenario can test five weeks at once.
- Expect 'what would you check next?' style questions.
- The thread gives you the order of operations.

Weeks 1-7 at a Glance

Concept and trigger, side by side

Week	Core concept	Data pattern that calls for it
1-2	Returns, stationarity, unit roots	always first — decide $I(0)$ vs $I(1)$
3-4	ARMA / ARIMA / SARIMA	a stationary (or differenced) mean, maybe seasonal
5	ARCH / GARCH	flat return ACF, clustered squared returns
6	VAR, Granger, IRF/FEVD	several interacting stationary series
7	Cointegration, ECM	$I(1)$ series sharing a long-run equilibrium

If you can fill the third column from a data description, you are ready.

Common Exam Traps

The mistakes that cost easy marks

The classic errors

- Regressing $I(1)$ levels and trusting a high R-squared.
- Reading Granger causality as true causation.
- Fitting ARIMA to returns expecting a tradable signal.

The right reflexes

- Check stationarity first — always.
- Say 'predicts', not 'causes', for Granger.
- Returns are for VARIANCE modeling, not mean forecasting.

A Regression That Looks Too Good

Weeks 2 and 7

You regress one stock's price on another's and get an R-squared of 0.95. A colleague concludes they are strongly related and wants to trade the relationship. State your concern and the test that settles it.

Think about

- Are the two price series stationary, or $I(1)$?
- What does a high R-squared between $I(1)$ series usually mean?
- How do you tell a real relationship from a spurious one?

The trap

- A high R-squared alone proves nothing here.
- It is the classic spurious-regression warning.
- The answer is on the next slide.

Spurious, Unless Cointegrated

The worked solution

The concern

- Both prices are almost certainly $I(1)$ (unit roots).
- Regressing $I(1)$ on $I(1)$ gives a spuriously high R-squared (Week 2).
- So the 0.95 is likely a statistical illusion, not a relationship.

The test that settles it

- Confirm each series is $I(1)$ (ADF/KPSS).
- Test the regression RESIDUAL for stationarity (Engle-Granger).
- Stationary residual $\rightarrow$ cointegrated (real, tradable); if not $\rightarrow$ spurious.

Reading a Return Series

Weeks 1, 3, and 5

A daily equity return series has an ACF near zero at all lags, but its SQUARED returns are strongly autocorrelated, and volatility jumps more after price drops than after rises. Which model would you fit, and why?

Think about

- What does a flat return ACF tell you about the mean?
- What does autocorrelation in squared returns indicate?
- What does the asymmetry after drops imply?

The clues

- Three separate hints point to three modeling choices.
- Each maps to a specific tool from Week 5.
- The answer is on the next slide.

A GARCH With Leverage and Fat Tails

The worked solution

Reading the clues

- Flat return ACF → the mean is unpredictable; don't fit ARIMA.
- Autocorrelated squared returns → volatility clustering (ARCH effects).
- Bigger jumps after drops → the leverage effect.

The model

- A GARCH-family model for the variance, with a trivial mean.
- GJR-GARCH or EGARCH to capture the asymmetry.
- Student-t innovations for the fat tails.

Format, Coverage, and Tips

What to expect and how to prepare

Coverage

- All of Weeks 1-7, weighted toward the connected thread.
- Concept identification, model choice, and interpretation.
- Reading plots (ACF/PACF, IRF, spreads) is fair game.

Tips

- Always state the stationarity check first.
- Justify EVERY model choice with the data pattern.
- Distinguish prediction from causation, spurious from cointegrated.

How to Prepare

Five focused steps

1 · Thread

the story

Retrace why
each week
follows the last.

2 · Tree

model choice

Practice routing
data to a
model.

3 · Plots

read them

Interpret
ACF/PACF, IRF,
and spreads by
eye.

4 · Traps

avoid

Drill spurious vs
cointegrated,
Granger caveat.

5 · Pairs

one example

Trace the pairs
trade through
all seven weeks.

Master the transitions and the decision tree, and the details fall into place.

Are You Ready?

Three questions that test the whole thread

Q1	Given a new series, what is the very first thing you check?	Stationarity ($I(0)$ vs $I(1)$)
Q2	Returns are unpredictable in the mean. So what do you model?	The variance (GARCH)
Q3	Two $I(1)$ series, stationary spread. What framework, and what trade?	Cointegration → pairs

If these come easily, you have the thread — and the midterm.

Three Things to Remember

Week 8 in one screen

1

State space is the unifying framework

Parts 1-2: a hidden state seen through noise, estimated by a predict-update cycle — filtering, smoothing, forecasting, and time-varying parameters, all in one language.

2

Part I is one connected story

Part 3: model the mean, then the variance, then jointly, then the long-run link. The decision tree and the pairs thread tie all seven weeks together.

3

Know the concept AND the trigger

For the midterm, for every tool know what it does and the data pattern that calls for it. Start every answer with the stationarity check.

Next — Week 9: the Kalman filter, the optimal engine behind everything in Part II.

★ WEEK 8 · PART 3 · PRACTICE

The Midterm, Worked by Hand

Diagnose a Return Series; Trade a Pair Across Weeks 6-8

Two exam-style problems that use the whole first-half toolkit

Part 3's synthesis, with the arithmetic shown. We work two integrative problems by hand - reading a return series for mean, autocorrelation, and volatility clustering, then building a pairs trade from cointegration to a live Kalman hedge.

3P

PART 3 · APPLICATIONS

Two Integrative Problems, by Hand

Case A: diagnose one return series - stationarity, an ARMA signal, and an ARCH signal - and name the model. Case B: run a pair from cointegration to z-score to a Kalman hedge. Then a Claude Code lab.

What the First Half Gave You

Now we use all of it - with numbers

1

Model the mean, then the variance

Stationarity (W1-2), ARMA for the mean (W3-4), GARCH for volatility (W5). Reading a return series uses all three.

2

Go joint, then find equilibrium

VAR and Granger (W6), cointegration and the ECM (W7). This is where a tradable pair comes from.

3

Then make the hidden visible

State space and the Kalman filter (W8) track hidden states - like a hedge ratio that drifts. We tie it all into two problems.

Everything here is a calculator exercise: one return series, one pair, the whole first-half toolkit.

How These Cases Work

Exam-style problems, worked step by step

Each case has three steps

- Start from raw prices or returns.
- Compute each diagnostic by hand.
- Then name the model or the trade.

What you should watch

- How one series reveals two structures.
- How a pair goes from data to a live trade.
- Which week each number comes from.

Diagnose a Return Series

Weeks 1-2, 3, and 5 in one question

A classic exam question: here are eight returns - what model do they need? We compute three diagnostics by hand: the mean and variance (is it stationary?), the lag-1 autocorrelation (is there an ARMA signal?), and the autocorrelation of squared returns (is there volatility clustering?)

The three checks

- Stationarity: mean and variance (W1-2).
- Mean structure: lag-1 autocorrelation (W3).
- Variance structure: squared-return ACF (W5).

What we'll do by hand

- Compute the mean, variance, and sd.
- Compute the lag-1 autocorrelation.
- Compute the ACF of squared returns.

The Return Series

Eight days to diagnose

day	1	2	3	4	5	6	7	8
return (%)	+0.5	-0.4	+0.3	-0.6	-1.7	+1.9	-1.6	+1.8

Notice two things by eye: the signs tend to alternate (mean reversion), and the series is quiet early, then loud - a clustering hint.

Mean, Variance, Stationarity

The Week 1-2 check

$$\begin{aligned}\text{mean} &= \frac{1}{8} \sum r_t = 0.03 \approx 0 \\ \text{variance} &= \frac{1}{7} \sum (r_t - \bar{r})^2 = 1.88, \quad \text{sd} = 1.37 \\ &\Rightarrow \text{mean near zero, finite variance: looks stationary (W1-2)}\end{aligned}$$

Mean near zero and a finite, stable variance: nothing screams non-stationary, so the classical toolkit applies.

Lag-1 Autocorrelation

The Week 3 check - is the mean predictable?

$$\rho_1 = \frac{\sum_t (r_t - \bar{r})(r_{t-1} - \bar{r})}{\sum_t (r_t - \bar{r})^2} = -0.65 \Rightarrow \text{strong mean reversion: fit AR / ARMA (W3)}$$

The number

- The lag-1 autocorrelation is -0.65.
- Strongly negative - today reverses yesterday.
- The alternating signs made this visible.

The verdict

- This is real mean structure, not noise.
- An AR(1) with a negative coefficient fits.
- So the mean needs an ARMA model (W3).

Squared-Return Autocorrelation

The Week 5 check - is the variance predictable?

$$\rho_1^{(r^2)} = \frac{\sum_t (r_t^2 - \bar{r^2})(r_{t-1}^2 - \bar{r^2})}{\sum_t (r_t^2 - \bar{r^2})^2} = +0.62 \Rightarrow \text{volatility clusters: fit GARCH (W5)}$$

The number

- The squared-return ACF is +0.62.
- Positive - big moves follow big moves.
- That is volatility clustering.

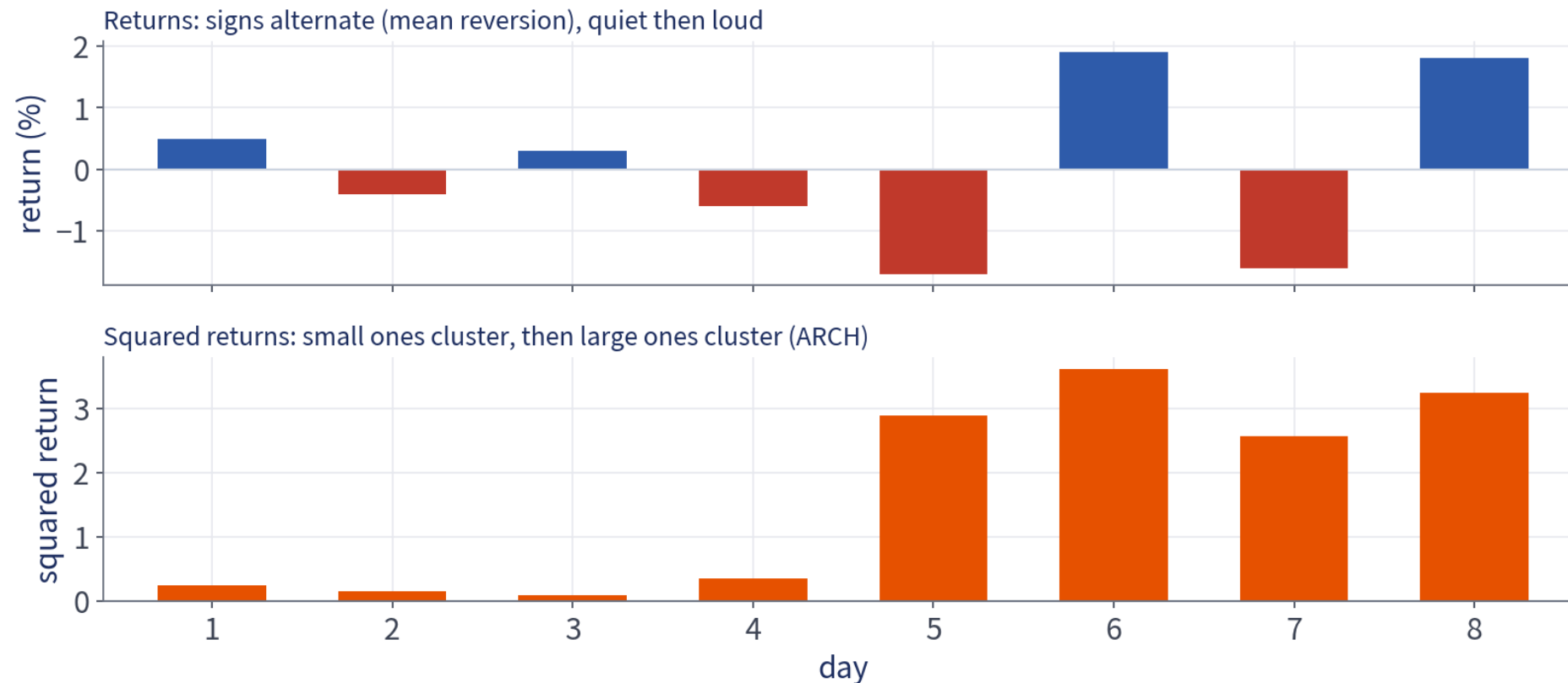
The verdict

- The variance is predictable, even if the sign isn't.
- This calls for an ARCH / GARCH model.
- So the variance needs GARCH (W5).

One Series, Two Structures

The returns (top) alternate in sign - the mean reversion an AR model captures. The squared returns (bottom) are quiet early and loud late - the volatility clustering GARCH captures. One series needs both a mean model and a variance model

Case A: one series, two structures - mean reversion and volatility clustering



The Full Diagnosis

What model does this series need?

What we found

- Stationary: mean ~ 0 , stable variance.
- Mean reverting: lag-1 autocorr -0.65.
- Clustered volatility: squared ACF +0.62.

What to fit

- An AR(1) or ARMA for the mean (W3).
- A GARCH for the variance (W5).
- Together: an ARMA-GARCH model.

A Pair, From Data to a Live Hedge

Weeks 6-8 in one question

The second integrative problem: two stocks that move together. We find the hedge ratio from cointegration (W7), turn the spread into a z-score entry (W1-3), then keep the hedge current with one Kalman update (W8). One pair walks through the whole second half of the first eight weeks

The three steps

- Cointegration: the hedge ratio and spread (W7).
- z-score: the entry signal (W1-3).
- Kalman: keep the hedge live (W8).

What we'll do by hand

- Compute beta from covariance and variance.
- Standardize today's spread into a z-score.
- Update beta with one new return pair.

The Hedge Ratio

Cointegration sets the pair (W7)

$$\beta = \frac{\text{Cov}(A,B)}{\text{Var}(B)} = \frac{0.074}{0.083} = \mathbf{0.89} \Rightarrow \text{spread} = A - 0.89 B \quad (\text{W7})$$

The calculation

- $\text{beta} = \text{Cov}(A,B) / \text{Var}(B) = 0.89$.
- It's the slope that best hedges A with B.
- The spread is A minus 0.89 times B.

Why it matters

- The spread is what you actually trade.
- A good hedge ratio makes it mean-reverting.
- This is the cointegration idea, applied.

The Entry Signal

A z-score triggers the trade (W1-3)

$$z = \frac{\text{spread}_{\text{today}} - \mu}{\sigma} = +1.8 \Rightarrow |z| > 1.5 : \text{spread is rich, SHORT it}$$

The calculation

- Standardize today's spread: $z = +1.8$.
- That's 1.8 standard deviations rich.
- Beyond the 1.5 sd band we set.

The action

- A rich spread should revert - so short it.
- Short A, long 0.89 of B, market-neutral.
- Exit as the spread returns toward its mean.

Keep the Hedge Live

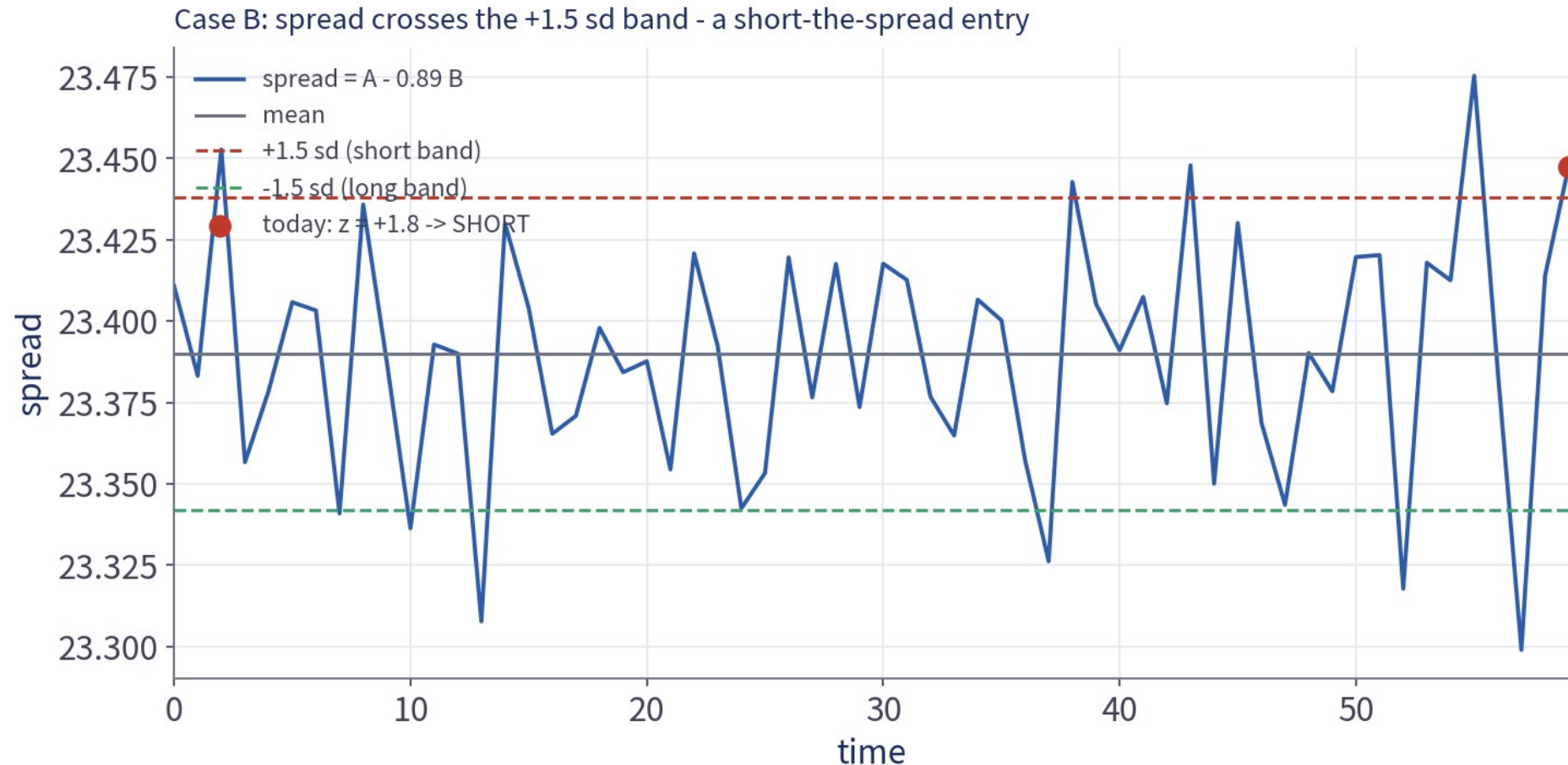
One Kalman update of beta (W8)

$$e = r_A - \beta r_B = 1.3 - 0.89(1.0) = 0.41$$
$$K = \frac{P r_B}{r_B^2 P + \sigma_\varepsilon^2} = \frac{0.021}{0.171} = 0.12, \quad \beta \leftarrow 0.89 + 0.12(0.41) = \mathbf{0.94} \text{ (W8)}$$

A new return pair with an innovation of 0.41 and a gain of 0.12 nudges the hedge ratio from 0.89 to 0.94 - the pair is kept in tune as it drifts.

The Spread and the Entry

The spread A minus $0.89 B$ oscillates around its mean. Today it crosses the $+1.5$ standard-deviation band at $z = +1.8$ - the short-the-spread entry. The Kalman filter keeps the 0.89 hedge ratio current as the relationship drifts



The Full Trade

Which week did each job

The trade

- Short the spread at $z = +1.8$.
- Hedge A with 0.94 of B, kept live.
- Exit as the spread reverts to its mean.

Which week did what

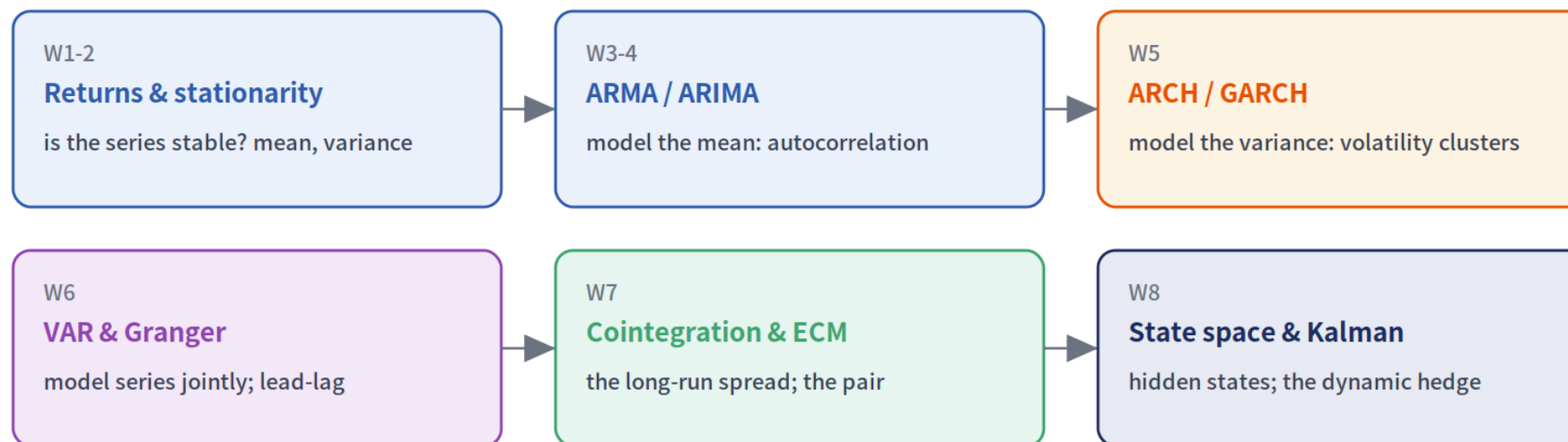
- W7 cointegration: the hedge ratio and spread.
- W1-3: the z-score entry signal.
- W8 Kalman: the drifting hedge, kept current.

The First-Half Toolbox, at a Glance

Both cases drew on the same eight-week arc: stationarity, ARMA, GARCH, VAR, cointegration, and state space. Read a series to pick a mean and variance model; build a pair to trade it - with the pairs thread tying every week together

The First-Half Toolbox, at a Glance

Eight weeks, one connected story: model the mean, find it is unpredictable, model the variance, go joint, then hidden.



The pairs-trading thread ties them together

Cointegration finds the pair (W7); a z-score triggers entry (W1-3); GARCH sizes it (W5); Kalman keeps the hedge current (W8).

How to Attack a Midterm Question

Diagnose, then decide

The habit

- Always check stationarity first.
- Separate the mean from the variance.
- Say which week's tool each finding calls for.

The payoff

- A messy series becomes a clear model choice.
- A pair of prices becomes a defined trade.
- You show you can connect the weeks.

Reproduce Both Diagnostics

Direct Claude Code to check your arithmetic

GOAL Verify the return-series diagnosis and the pairs pipeline, then run them on real data.

[INPUT] Prompt · Claude Code

Write and run `part3_practice.py` that:

CASE A - diagnose:

- * take the 8 returns; print mean, variance, and sd;
- * print the lag-1 autocorrelation of returns;
- * print the lag-1 autocorrelation of squared returns;
- * name the model (ARMA for mean, GARCH for var).

CASE B - the pair:

- * from the price table, compute $\beta = \text{Cov}/\text{Var}$;
- * form the spread and today's z-score;
- * do one Kalman update of β .

Match every printed number to the slides.

Check against the slides

- mean ~ 0 , sd 1.37.
- return autocorr -0.65.
- squared autocorr +0.62.
- beta 0.89 $\rightarrow$ 0.94, $z = 1.8$.

Deliverable: `part3_practice.py` whose printed numbers match both worked diagnoses.

Three Questions Before the Exam

If you can answer these, you're ready

Q1 What does a negative return autocorrelation suggest?

**Mean reversion - fit
ARMA**

Q2 What does a positive squared-return ACF suggest?

**Volatility clustering -
fit GARCH**

Q3 Which weeks build a pairs trade?

**Cointegration (W7),
z-score (W1-3),
Kalman (W8)**

All three come from the return-series diagnosis and the pairs pipeline.

Three Pitfalls to Avoid

Where midterm answers slip

Common mistakes

- "Returns are unpredictable, so skip the ACF."
- "One model handles mean and variance."
- "A fixed hedge ratio is fine forever."

The reality

- Check both return and squared-return ACF.
- The mean and the variance need separate models.
- A drifting hedge needs a Kalman update.

Two Problems to Remember

Part 3 applications in one screen

1

Diagnose a series in three checks

Mean ~ 0 with stable variance (stationary), a lag-1 autocorrelation of -0.65 (mean reversion, fit ARMA), and a squared-return ACF of +0.62 (volatility clustering, fit GARCH). One series, two structures.

2

Build a pair from data to a live hedge

Cointegration gave $\beta = 0.89$ and the spread; today's z-score of +1.8 triggered a short; one Kalman update kept the hedge current at 0.94. Weeks 7, 1-3, and 8, in one trade.

3

Connect the weeks

The midterm rewards showing how the tools fit together - stationarity, ARMA, GARCH, cointegration, and Kalman - with the pairs thread running through them all.

Next — Week 9 and the second half: the Kalman filter deepens, and new state-space models arrive.